Transformations Based on Continuous Piecewise-Affine Velocity Fields

Oren Freifeld, *Member, IEEE,* Søren Hauberg, *Member, IEEE,* Kayhan Batmanghelich, *Member, IEEE,* and Jonn W. Fisher III, *Member, IEEE,*

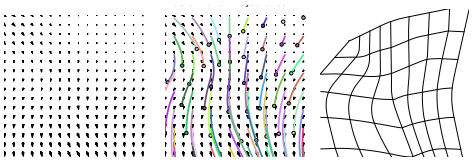
Abstract—We propose novel finite-dimensional spaces of well-behaved $\mathbb{R}^n \to \mathbb{R}^n$ transformations. The latter are obtained by (fast and highly-accurate) integration of continuous piecewise-affine velocity fields. The proposed method is simple yet highly expressive, effortlessly handles optional constraints (*e.g.*, volume preservation and/or boundary conditions), and supports convenient modeling choices such as smoothing priors and coarse-to-fine analysis. Importantly, the proposed approach, partly due to its rapid likelihood evaluations and partly due to its other properties, facilitates tractable inference over rich transformation spaces, including using Markov-Chain Monte-Carlo methods. Its applications include, but are not limited to: monotonic regression (more generally, optimization over monotonic functions); modeling cumulative distribution functions or histograms; time-warping; image warping; image registration; real-time diffeomorphic image editing; data augmentation for image classifiers. Our GPU-based code is publicly available.

1 Introduction

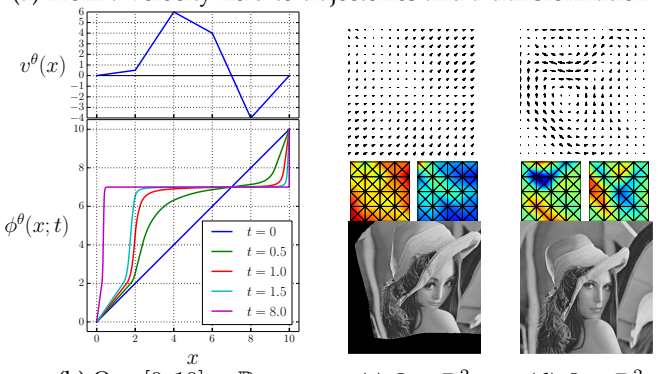
Diffeomorphisms are important in many fields such as computer vision, medical imaging, graphics, and robotics. Unfortunately, current representations of *highly-expressive* diffeomorphism spaces are overly complicated. Thus, despite their potential and mathematical beauty, their applicability is limited, especially in large datasets or when computing time is restricted. Moreover, in such spaces, owing to their complexity, using powerful inference tools, *e.g.* Monte Carlo Markov Chain (MCMC), still presents challenges, although some encouraging recent progress has been made in this active field of research (*e.g.*, [1], [2], [3]). Lastly, seemingly-formidable mathematical preliminaries render these spaces accessible to only a small group of geometry experts. This hinders the exchange of ideas between communities and unnecessarily limits the potential impact of diffeomorphism-based methods; *e.g.*, while certain machine-learning areas can benefit from such methods, little work has been done in this direction, partly since practical computational tools have yet to become available.

Motivated by practicalities of probabilistic modeling and statistical inference as well as a desire to make diffeomorphisms broadly accessible, in this work, which expands [4], we propose a *representation* that (as we will show in Sec. 4) combines simplicity, expressiveness, and efficiency. Particularly, *we propose new spaces of transformations that are based on (fast, highly-accurate) integration of Continuous Piecewise-Affine (CPA) velocity fields*. Importantly, as we will show, their benefits go beyond speed and accuracy.

Possible applications of the proposed representation are numerous, as we demonstrate here with: image editing and shape manipulation; unconstrained optimization over monotonic functions; modeling of Cumulative Distribution Functions (CDFs) and histograms with order-preserving geometry; time warping; image registration; landmark-based image warping/animation; "prettifying" results of (non-



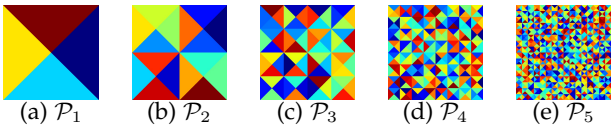
(a) From a velocity field to trajectories and a transformation



(b) $\Omega = [0, 10] \subset \mathbb{R}$ (c) $\Omega \subset \mathbb{R}^2$ (d) $\Omega \subset \mathbb{R}^2$

Fig. 1: (a) Integration of sufficiently-nice velocity fields is widely used to generate well-behaved nonlinear transformations. The choice of using CPA velocity fields, among other benefits, reduces computational costs, increases integration accuracy, and simplifies modeling and inference. A CPAB transformation, $\boldsymbol{x} \mapsto \boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, t)$, is one that is *based* (via integration) on a CPA velocity field, $\boldsymbol{v}^{\boldsymbol{\theta}}$. (b) A 1D example. (c-d) Two 2D examples, where in (d) there are also additional constraints. Top row: a continuously-defined $\boldsymbol{v}^{\boldsymbol{\theta}}$ in select locations. Middle: Visualizing the horizontal ($\boldsymbol{v}_h^{\boldsymbol{\theta}}$, left) and vertical ($\boldsymbol{v}_v^{\boldsymbol{\theta}}$, right) components as heat maps highlights the CPA property; blue=$-\lambda$, green=0, and red=λ where λ=$\max_{\boldsymbol{x}\in\Omega} \max(|\boldsymbol{v}_h^{\boldsymbol{\theta}}(\boldsymbol{x})|, |\boldsymbol{v}_v^{\boldsymbol{\theta}}(\boldsymbol{x})|)$. Bottom: $I_{\text{src}} \circ \boldsymbol{\phi}^{\boldsymbol{\theta}}(\cdot, 1)$.

- *O. Freifeld, K. Batmanghelich, and J.W. Fisher III are with the Computer Science and Artificial Intelligence Lab, Massachusetts Institute of Technology, Cambridge MA 02319, USA.*
  E-mail: {freifeld, kayhan, Fisher}@csail.mit.edu
- *S. Hauberg is with the Section for Cognitive Systems, DTU Compute, Kongens Lyngby, Denmark 2800.*
  E-mail: sohau@dtu.dk



(a) $\mathcal{P}_1$ (b) $\mathcal{P}_2$ (c) $\mathcal{P}_3$ (d) $\mathcal{P}_4$ (e) $\mathcal{P}_5$

Fig. 2: Several type-I tessellations of a 2D region.

(a) $\mathcal{P}_i$ (b) $\mathcal{P}_{ii}$ (c) $\mathcal{P}_{iii}$ (d) $\mathcal{P}_{iv}$ (e) $\mathcal{P}_v$

Fig. 3: Several type-II tessellations of a 2D region.

diffeomorphic) dense-correspondence tools. Moreover, we recently used the proposed representation as a key component in a novel (and *learned*) data-augmentation scheme [5], improving the results of image classifiers. Finally, our code is available at github.com/freifeld/cpabDiffeo.

2 Related Work

Pattern theory and differential geometry. Representing objects via transformations acting on them is a *Pattern-Theoretic* cornerstone [6]. Our work is influenced by many impressive works in this field, primarily in the geometry-oriented areas of computer vision and medical imaging. Due to space limits, we can mention only a few: [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19]. *Most* of works are based on complicated, *usually* ∞-dimensional, spaces, and the associated representations and computations are, in practice, discretized and/or otherwise approximated. We take a more practical approach and start from a finite-dimensional space, in which discretizing the representation is unneeded, while computations require no approximations in the 1D case and almost no approximations in higher dimensions. The result is a simple, efficient, and practical machinery for working with a rich space of diffeomorphisms. The rapidness and high accuracy of Algorithm 1 (see Sec. 4), together with the fact that the machinery is easily implemented on GPU, let us evaluate transformations fast. This, in turn, together with the fact that the representation effortlessly supports coarse-to-fine analysis and the use of smoothness priors, allows the use of general-purpose powerful inference methods such as MCMC. See also other recent approaches for MCMC on diffeomorphisms, *e.g.*, [1], [2], [3].

In applications involving landmark pairs, most methods above can deal with only a small number of landmarks as their inference complexity grows super-linearly with the number of landmarks. Consequently, they cannot leverage the success of popular tools for dense-correspondence extraction (*e.g.*, [20], [21]), highlighting a disconnect with the larger computer-vision community. Our algorithms have linear complexity and are (embarrassingly) parallelizable, yielding sub-linear running times in practice.

Note that our representation, despite the fact it involves tessellations, is *not* not based on control points (such as, *e.g.*, [2], [19]); *e.g.*, in cases where we either impose volume preservation or use type-II tessellations (see Sec. 4), one cannot simply define arbitrary velocities at the tessellation vertices as there are too few degrees of freedom.

PA and CPA affine maps in computer vision. Cootes et al. [22] use PA transformations. The latter, while simple and efficient, are neither continuous nor invertible. Several other works use CPA transformations (*e.g.* [23]; see also [24]), sometimes with additional constrains (*e.g.* [25]). Common to those works is the *direct* use of PA (or CPA) maps as transformations. Those are not differentiable (and are often not invertible), hence not diffeomorphisms. In contrast, by *integrating* CPA *velocity fields*, we obtain transformations – which themselves are *not* PA (hence not CPA) – that are (orientation-preserving) diffeomorphisms. Lin et al. [26] work with CPA fields but, unlike us who actually solve Eqn. (3) (to exploit its link, via integration, to transformations), they use it only as a regression model.

Closer to ours is the elegant *log-Euclidean polyaffine* method proposed by Arsigny et al. [27] (see also [28], [29]) that, similar to ours, uses finitely-many affine building blocks to build flexible velocity fields. They use a spatial averaging with smoothly-decaying weights of the blocks to ensure a smooth velocity field whose integration yields trajectories that define a diffeomorphism. However, as their integral has no closed form they must use an approximation throughout; *i.e.* they must numerically approximate the entire trajectory. This is computationally demanding and can cause substantial approximation errors. Their method is based on approximating the integral of a weighted sum of affine velocity fields via a weighted sum of the affine diffeomorphisms associated with these fields. To reduce errors, they divide the field by a large $2^{\#\text{steps}}$ as the approximation holds only for near-zero velocity. To keep the number of steps not too large, they smartly generalize the scaling-and-squaring method. The result is, however, still exact only if a single affine component is used. For expressiveness, however, a larger number is needed, and accuracy drops. It is thus unsurprising they state they focus on a small number of affine components, and that to achieve reasonable timings, they use their method only in the last stage of inference; till then they resort to a non-diffeomorphic fusion. In contrast, we use affine building blocks in a different way; *i.e.*, we use them in a piecewise manner, but, using linear constraints, force them to yield everywhere-continuous velocity fields. This seemingly-subtle difference has profound implications. In 1D, it yields a closed-form integration; in higher dimensions it lets us integrate almost the entire trajectory in closed form using large (hence few) steps which are exact, and only in small portions of the trajectory do we resort to a numerical solver. This virtually eliminates numerical issues and allows construction of substantially more expressive transformations; *e.g.* Arsigny *et al.* [27] report using 7 affine blocks, while we routinely use tens or hundreds while neither accuracy nor computational cost becomes an issue. Though this indirect comparison of *integration methods* (as their method is inapplicable to our velocity fields and vice versa, a direct comparison is impossible) favors ours, it is just part of the story. Our *representation* has additional advantages over theirs, including simplicity, better suitability for a GPU implementation, trivial handling of boundary constraints and volume preservation, and simpler encoding of statistical priors and coarse-to-fine analysis.

Discrete representations and approximations. Allassonnire et al. [13] efficiently *approximate* diffeomorphisms.

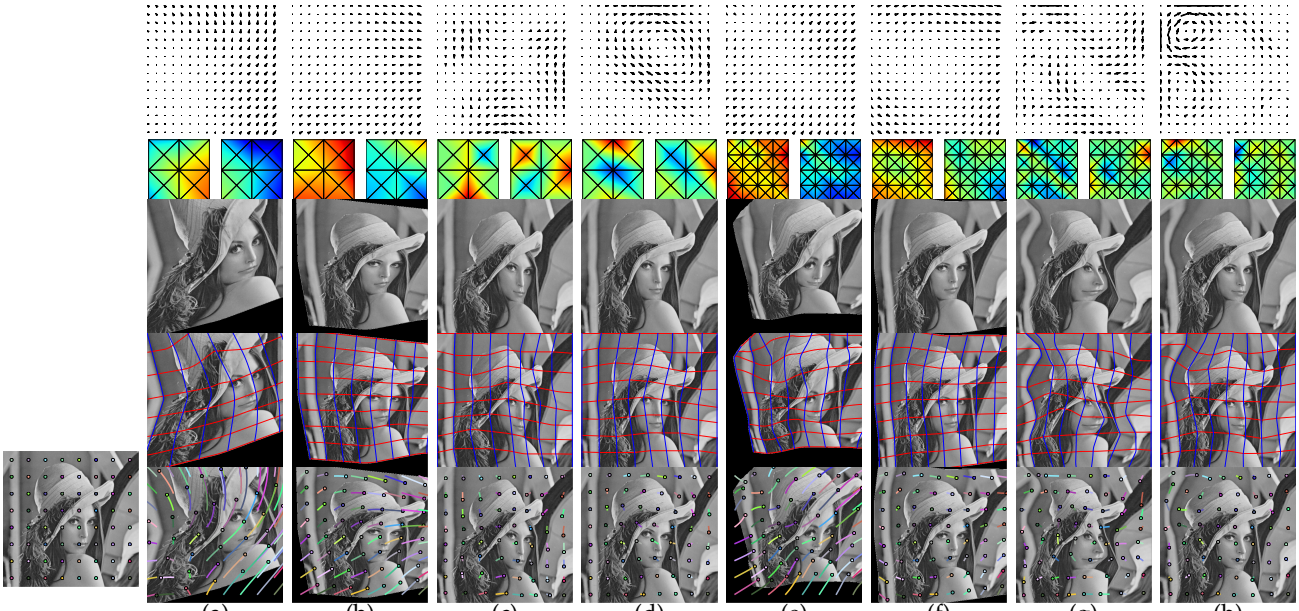


Fig. 4: Samples from the prior (Sec. 4). (a-h) The top 3 rows echo those in Fig. 1c and Fig. 1d. The 4^{th} row shows a deformed grid overlaid on the image. The 5^{th} row shows select *trajectories*. Note that the trajectories and transformations are differentiable (hence continuous) but *not* piecewise affine. The tessellation in (e-h) is a *refinement* of the one in (a-d). (a) & (e): $T^{\boldsymbol{\theta}} \in M$. (b) & (f): $T^{\boldsymbol{\theta}} \in M^{\text{vp}}$. (c) & (g): $T^{\boldsymbol{\theta}} \in M^{\partial}$. (d) & (h): $T^{\boldsymbol{\theta}} \in M^{\partial,\text{vp}}$. See Sec. 4 for details.

Unlike our transformations, their non-differentiable transformations are not diffeomorphisms. While we focus on a general-purpose representation, diffeomorphic demons [16] is a registration method, popular due to its speed, based on discretely-defined fields. As its authors note, these may be inconsistent with a diffeomorphic framework and may not preserve orientation. They also cannot easily impose volume preservation, though some success was reported [30]. Also, a computer representation of a discrete-field sequence needs plenty of memory. These issues can be obviated by adapting their method to use our compact and continuously-defined fields. More generally, approximations based on discretely-defined fields and/or discrete diffeomorphisms are widely used, *e.g.*, in medical imaging [16], [31], robotics [32], [33], [34], geometric modeling [35] and fluid dynamics [36]. Unlike these works, both our fields and transformations are continuously-defined and more compact.

Statistics on manifolds and tangent spaces. Like many authors (including of some aforementioned works), we handle the nonlinearity of a space via the linearity of the tangent space at the identity [11], [27], [29], [37], [38], [39], [40], [41]. Other tools (not explored here) for statistics on manifolds that either use other tangent spaces or work on the manifold itself [42], [43], [44], [45], [46], [47], [48] may also be applied to our spaces. Particularly, parallel-transport tools may be especially relevant here [49], [50], [51], [52], [53], [54], [55].

CDF/histogram modeling. In modeling distributions, it is better to work with *cummulative distribution functions (CDFs)* over densities since the latter might not exist and inter-density L_p (or sphere-based) distances can be arbitrarily large even if their *probability measures* are essentially the same. One approach to CDF representation uses p-Wasserstein spaces, usually $p \in \{1, 2\}$, the $p = 1$ case is tied to Earth Mover's Distance [56]. A limitation of this approach is that it needs bounded p^{th}-moments and hard computations. 1-Wasserstein methods also lack easy synthesis of new points that are valid histograms/CDFs. While it is less of an issue for spherical methods [57], they suffer from two issues. 1) They do not respect the ordering of the bins or of $\mathbb{R}$. While, for histograms, bin ordering is *sometimes* immaterial, the ordering of $\mathbb{R}$ matters. 2) Large moves on the sphere lead to CDFs/histograms with negative values. The problems above do not exist in our representation.

Image warping and shape manipulation. Related to ours are works on image warping ([58], [59], [60], [61], [62]) and shape manipulation (*e.g.*, [63]). Unlike most methods, ours is fast, invertible and handles constraints effortlessly.

Benefits of the Proposed Representation To summarize, existing spaces of diffeomorphisms offer only subsets of the following list: 1) high expressiveness; 2) ease of implementation; 3) modest mathematical preliminaries (basic linear algebra and ODE); 4) ease of handling optional constraints (*e.g.*, volume preservation); 5) convenient modeling choices (coarse to fine, easy-to-use smoothness priors); 6) finite dimensionality; 7) fast *and* highly-accurate computations. These benefits, especially the last three, render more tractable the use of inference tools that are usually too expensive in the context of rich diffeomorphisms.

3 High-Level Summary

The section provides a summary of the proposed representation, laying the ground for the formal treatment (Sec. 4). Let Ω be either $\mathbb{R}^n$ or a certain type, to be defined later, of a proper subset of $\mathbb{R}^n$ (*i.e.*, $\Omega \subsetneq \mathbb{R}^n$). A popular way to obtain a diffeomorphism, $T : \Omega \to \Omega$, is via the integration of velocity fields; see Fig. 1a. The choice of velocity-field family affects the dimensionality, structure, and expressiveness of the space of the resulting diffeomorphisms, as well as the

$\mathcal{P}$	$N_{\mathcal{P}}$	N_v	$D = 6N_{\mathcal{P}}$	$d = 2N_v$
$\mathcal{P}_1$	4	5	24	10
$\mathcal{P}_2$	16	13	96	26
$\mathcal{P}_3$	64	41	384	82
$\mathcal{P}_4$	256	145	1536	290
$\mathcal{P}_5$	1024	1025	6144	1090

TABLE 1: Values of $N_{\mathcal{P}}$, N_v, $D = \dim(\mathcal{V}'_{\Omega,\mathcal{P}})$, and $d = \dim(\mathcal{V}_{\Omega,\mathcal{P}})$ for the $\mathcal{P}$'s shown in Fig. 2. See also Sec. 4.

accuracy and computational complexity of the integration. Thus, this choice crucially affects which probabilistic models can be used and the tractability of the statistical inference.

CPA velocity fields. We base our representation on spaces of $\Omega \to \mathbb{R}^n$ CPA velocity fields (Fig. 1). The term 'piecewise' is w.r.t. a certain *tessellation* (Sec. 4.1), denoted by $\mathcal{P}$. Let $\mathcal{V}_{\Omega,\mathcal{P}}$ be such a space. While $\mathcal{V}_{\Omega,\mathcal{P}}$ depends on Ω and $\mathcal{P}$, we will usually notationally suppress these dependencies, and will just write $\mathcal{V}$. One appeal of these spaces is that they are *finite-dimensional* and *linear* (although their elements, *i.e.*, the velocity fields, are usually nonlinear). Let $d = \dim(\mathcal{V})$. The spaces $\mathbb{R}^d$ and $\mathcal{V}$ are identified with each other (as we will explain in Sec. 4.2, Eqn. (11)), where every $\boldsymbol{\theta} \in \mathbb{R}^d$ is identified with exactly one element of $\mathcal{V}$, denoted by $\boldsymbol{v}^{\boldsymbol{\theta}}$, and vise versa. Symbolically, we write

$$\boldsymbol{\theta} \leftrightarrow \boldsymbol{v}^{\boldsymbol{\theta}} \quad \text{where} \quad \boldsymbol{v}^{\boldsymbol{\theta}} \in \mathcal{V}\,,\ \boldsymbol{\theta} \in \mathbb{R}^d\,. \tag{1}$$

Likewise, $\boldsymbol{\theta} + \boldsymbol{\theta}' \leftrightarrow \boldsymbol{v}^{\boldsymbol{\theta}} + \boldsymbol{v}^{\boldsymbol{\theta}'} \triangleq \boldsymbol{v}^{\boldsymbol{\theta}+\boldsymbol{\theta}'}$ and $\alpha\boldsymbol{\theta} \leftrightarrow \alpha\boldsymbol{v}^{\boldsymbol{\theta}} \triangleq \boldsymbol{v}^{\alpha\boldsymbol{\theta}}$ where $\boldsymbol{\theta}, \boldsymbol{\theta}' \in \mathbb{R}^d$ and $\alpha \in \mathbb{R}$. Note that d depends on $\mathcal{P}$ (and typically grows with n). A finer $\mathcal{P}$ implies a higher d and richer velocity fields and vice versa (Figs. 4, 2, and 3).

Remark 1. There are many finite-dimensional linear spaces of continuous velocity fields (*e.g.*, [27] or other spaces based on splines). We will show that CPA spaces, however, have additional useful properties in our context.

From CPA velocity fields to trajectories. Modulo a detail (to be explained in Sec. 4.4) related to the case $\Omega \subsetneq \mathbb{R}^n$, any continuous $\Omega \to \mathbb{R}^n$ velocity field, whether Piecewise-Affine (PA) or not, defines differentiable $\mathbb{R} \to \Omega$ *trajectories*. If $\boldsymbol{x} \in \Omega$ then $\boldsymbol{v}^{\boldsymbol{\theta}} \in \mathcal{V}$ defines a trajectory, $t \mapsto \boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, t)$, such that $\boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, 0) = \boldsymbol{x}$ and $\boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, t)$ solves the *integral equation*

$$\boxed{\boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, t) = \boldsymbol{x} + \int_0^t \boldsymbol{v}^{\boldsymbol{\theta}}(\boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, \tau))\, d\tau \ \text{ where } \boldsymbol{v}^{\boldsymbol{\theta}} \in \mathcal{V}}\,. \tag{2}$$

The equivalent ODE (with an initial condition $\boldsymbol{x}$) is

$$d\boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, t)/dt = \boldsymbol{v}^{\boldsymbol{\theta}}(\boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, t))\,. \tag{3}$$

Remark 2. Equation (2), whose unknown $\boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, \cdot)$ is both inside and outside the integral, should not be confused with the piecewise-quadratic $\Omega \to \mathbb{R}^n$ map, $\boldsymbol{y} \mapsto \int_{\mathbf{0}_{n\times 1}}^{\boldsymbol{y}} \boldsymbol{v}^{\boldsymbol{\theta}}(\boldsymbol{x})\, d\boldsymbol{x}$. The latter, a popular tool in computer-vision [24] and numerical analysis, is unrelated to our work. Particularly, both $\boldsymbol{x} \mapsto \boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, t)$ and $t \mapsto \boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, t)$ are *not* piecewise quadratic.

CPA-Based (CPAB) transformations. Modulo that detail, any continuous $\Omega \to \mathbb{R}^n$ velocity field, whether PA or not, defines a *transformation*; *i.e.*, a map whose input and output are viewed as points, not vectors. Letting $\boldsymbol{x}$ vary and fixing t, $\boldsymbol{x} \mapsto \boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, t)$ is an $\Omega \to \Omega$ transformation. Without loss of generality (Sec. 4), we may set $t = 1$ and define

$$T^{\boldsymbol{\theta}}(\cdot) \triangleq \boldsymbol{\phi}^{\boldsymbol{\theta}}(\cdot, 1), \quad \boldsymbol{\theta} \in \mathbb{R}^d\,. \tag{4}$$

$N_{\mathcal{P}}$	5	25	45	65	85	105
improvement [%]	2.99	12.56	11.26	9.98	3.29	3.73

TABLE 2: Accuracy: Improvement, in %, of Algorithm 1 ($N_{\text{steps}} = 10$, $n_{\text{steps}} = 10$) over the generic solver (100 steps). $\Omega = [0, 1]$ was tessellated into $N_{\mathcal{P}}$ equal-length intervals.

Since we integrate CPA velocity fields, we coin our transformations CPA-*Based* (CPAB).

Remark 3. CPAB transformations are *not* CPA (unless the CPA field is affine): while $T^{\boldsymbol{\theta}}$ is continuous, it is not PA.

The notation $T^{\boldsymbol{\theta}} = \exp(\boldsymbol{v}^{\boldsymbol{\theta}})$ indicates that $T^{\boldsymbol{\theta}}$ is defined by integrating $\boldsymbol{v}^{\boldsymbol{\theta}} \leftrightarrow \boldsymbol{\theta}$ for $t = 1$ (see Sec. 4.9). We let

$$M \triangleq \exp(\mathcal{V}) \triangleq \{\exp(\boldsymbol{v}^{\boldsymbol{\theta}}) : \boldsymbol{v}^{\boldsymbol{\theta}} \in \mathcal{V}\} \tag{5}$$

denote the space of CPAB transformations, where we notationally suppressed that both M and $\mathcal{V}$ depend on both Ω and $\mathcal{P}$ (*i.e.*, more formally we should write $M_{\Omega,\mathcal{P}}$). Note that M is nonlinear: *i.e.*, both $T^{\boldsymbol{\theta}} + T^{\boldsymbol{\theta}'}$ and $\alpha T^{\boldsymbol{\theta}}$ are usually not in M. The linearity of $\mathcal{V}$, together with $\exp : M \to \mathcal{V}$, however, provides a convenient way to handle this nonlinearity.

CPAB transformations are well behaved. *E.g.*, they are *diffeomorphisms*; moreover, $(T^{\boldsymbol{\theta}})^{-1} \in M$ and the inversion is simple. Appealingly, useful subsets of M (*e.g.*, volume-preserving CPAB transformations) are easily obtained via linear subspaces of $\mathcal{V}$ (we will return to this point in Sec. 4).

Remark 4. $\boldsymbol{x} \mapsto \boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, t = 1)$ should not be confused with (the non-diffeomorphism, parametric optical-flow-like representation) $\boldsymbol{x} \mapsto \boldsymbol{x} + \boldsymbol{v}^{\boldsymbol{\theta}}(\boldsymbol{x})$, the latter being only a *Taylor approximation* of the former. Thus, the way we utilize flexible parametrized vector fields is different from, *e.g.*, [64], [65].

Integration of CPA velocity fields. Integral equations usually lack analytic solutions. Since CPA velocity fields are Lipschitz continuous and almost-everywhere smooth, generic integration solvers are quite effective for them. However, we can do even better. *One of our contributions is showing that integration of such fields is given in either closed form ($n = 1$) or almost closed form ($n > 1$).* Besides its obvious pluses (accuracy, computing time), this analytic solution makes it easier to interpret the resulting trajectories/transformation and is key to the theorems in Sec. 4.

A specialized numerical solver. There is a shortcoming to that solution: as we will explain after Theorem 2, it requires tedious bookkeeping (if $n > 1$) and invoking certain routines (easy if $n = 1$ but hard if $n > 1$). This is especially a hurdle with GPU. We thus propose a practical alternative, *a specialized solver for integrating CPA velocity fields*, which is faster *and* more accurate than non-specialized solvers.

Convenient modeling, tractable inference. Smoothness priors on M are easy to build and use. As is common with nonlinear spaces of nonlinear transformations, the nonlinearity of $(\boldsymbol{\theta}, \boldsymbol{x}) \mapsto T^{\boldsymbol{\theta}}(\boldsymbol{x})$ prohibits closed forms for the posterior or Maximum Likelihood (ML) estimation. However, since our transformations are evaluated fast, we rapidly evaluate $\boldsymbol{\theta} \mapsto p(\text{data}|T^{\boldsymbol{\theta}})$. This, together with the relatively-modest dimensionality and ease of using priors, facilitates the use of inference methods that require multiple likelihood evaluations (e.g., MCMC). Lastly, spaces of CPA velocity fields support coarse-to-fine approaches.

Input: $\mathcal{P} = \{C_c\}_{c=1}^{N_\mathcal{P}}$; $(A_{1,\boldsymbol{\theta}}, \ldots, A_{N_\mathcal{P},\boldsymbol{\theta}})$; U = a sequence of N_{pts} points in Ω; $t > 0$; $N_{\text{steps}}, n_{\text{steps}} \in \mathbb{Z}^+$
Output: $\boldsymbol{\phi}^{\boldsymbol{\theta}}(U,t) \triangleq \{\boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x},t)\}_{\boldsymbol{x}\in U} \subset \Omega$

$\Delta_t \leftarrow t/N_{\text{steps}}$; $\delta_t \leftarrow \Delta_t/n_{\text{steps}}$
for $c \in \{1,..,N_\mathcal{P}\}$ **do in parallel**
  $T_{A_{c,\boldsymbol{\theta}},\Delta_t} \leftarrow \exp(\Delta_t \widetilde{A_{c,\boldsymbol{\theta}}})$
for $\boldsymbol{x} \in U$ **do in parallel**
  $\boldsymbol{x}_0 \leftarrow \boldsymbol{x}$
  for $i \in \{1,..,N_{\text{steps}}\}$ **do**
    $\widetilde{\boldsymbol{x}}_{\text{temp}} = [\,\boldsymbol{x}_{\text{temp}}^T\ 1\,]^T \leftarrow T_{A_{\gamma(\boldsymbol{x}_{i-1}),\boldsymbol{\theta}},\Delta_t}\widetilde{\boldsymbol{x}}_{i-1}$
    if $\gamma(\boldsymbol{x}_{i-1}) == \gamma(\boldsymbol{x}_{\text{temp}})$ **then**
      $\boldsymbol{x}_i \leftarrow \boldsymbol{x}_{\text{temp}}$ `// analytic update`
    else
      `// inter-cell bdry crossed`
      $\boldsymbol{x}_i \leftarrow$ genericODEsolver(ini. con.=$\boldsymbol{x}_{i-1}$, step size=δ_t, n_{steps}, $\boldsymbol{v}(\cdot) : \boldsymbol{x} \mapsto A_{\gamma(\boldsymbol{x}),\boldsymbol{\theta}}\widetilde{\boldsymbol{x}}$)
  $\boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x},t) \leftarrow \boldsymbol{x}_i$

Algorithm 1: Integrating $\boldsymbol{v}^\theta \in \mathcal{V}$. See text for details.

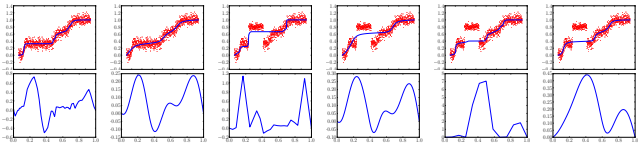


Fig. 5: Monotonic regression on synthetic data (top) by inferring CPA fields (bottom), defined over 100 equal-length cells. We used a squared loss in the 4 leftmost columns and a robust one (Geman-McClure) in the 2 others. Col. 1, 3, and 5 show ML solutions. Col. 2, 4 and 6 show MAP solutions with a smoothness prior. In all cases, the function is increasing, as obtained effortlessly from the representation.

4 The Mathematical Representation

Let Ω be a Cartesian product of n compact intervals; *i.e.*, Ω is a closed *n-orthotope*, also called a (closed) hyperrectangle. The lemmas/theorems below are proved in our supplemental material, henceforth referred to as **Sup. Mat.**

4.1 Tessellations

A (finite) tessellation, denoted by $\mathcal{P} = \{U_c\}_{c=1}^{N_\mathcal{P}}$ (where $N_\mathcal{P}$ is a positive integer), is a set of $N_\mathcal{P}$ closed subsets of Ω, also called *cells*, such that their union is Ω and the intersection of any pair of adjacent cells is their shared border (see Figs. 2 and 3). Henceforth we will always assume the cells are *convex n-polytopes* (namely, a subset of $\mathbb{R}^n$ whose sides are flat). We define 3 cell types. Type-I cells have $n+1$ vertices; *i.e.*, intervals (if $n = 1$), triangles ($n = 2$), tetrahedra ($n = 3$), etc. Type-II cells are *n-orthotopes* (*i.e.*, hyperrectangles). Note intervals fit both these types. Type-III includes all other convex n-polytopes. We say that $\mathcal{P}$ is of type I (respectively, II, III) if all its cells are of type I (II, III). A type-II tessellation is called *regular* if its cells are evenly spaced along each of the n dimensions. As we will show, type-I tessellations are special, having benefits possessed by neither type II nor III. Mathematically, there is no difference (for our purposes) between types II and III; all theoretical results below that apply to type-II cells also apply to type-III cells. Type-II cells, however, lead to faster computations and simpler implementations than both other types, especially when compared with type III. Thus, unless stated otherwise, we restrict our attention to types I and II. Let

$$\mathcal{P}_{\text{vert}} = \{\boldsymbol{\xi} : \boldsymbol{\xi} \text{ is a vertex of some } U_c \in \mathcal{P}\} \tag{6}$$

and let N_v denote its cardinality; *e.g.*, see Fig. 2 and Table 1. Henceforth we will view $\mathcal{P}_{\text{vert}}$ as an ordered N_v-tuple; the ordering is arbitrary, but assumed fixed. Let $N_\mathcal{P}{}' > N_\mathcal{P}$. An $N_\mathcal{P}{}'$-cell tessellation, $\mathcal{P}' = \{U_{c'}\}_{c'=1}^{N_\mathcal{P}{}'}$, is a *refinement* of $\mathcal{P}$, a relation denoted $\mathcal{P} \prec \mathcal{P}'$, if each $U_{c'} \in \mathcal{P}'$ is a (possibly-improper) subset of some $U_c \in \mathcal{P}$.

4.2 CPA Velocity Fields

Fix $\mathcal{P}$, let $\boldsymbol{x} \in \Omega$, and define the membership function $\gamma : \Omega \to \{1,\ldots,N_\mathcal{P}\}$, $\gamma : \boldsymbol{x} \mapsto \min\{c : \boldsymbol{x} \in U_c\}$. *I.e.*, if $\boldsymbol{x}$ is not on an inter-cell border, then $\gamma(\boldsymbol{x}) = c \iff \boldsymbol{x} \in U_c$, while a border point is (arbitrarily) assigned to the cell of the lower index.

Definition 1. A map, $f : \Omega \to \mathbb{R}^n$, is called PA (w.r.t. $\mathcal{P}$) if $\{f|_{U_c}\}_{c=1}^{N_\mathcal{P}}$ are *affine*; *i.e.*, $f(\boldsymbol{x}) = A_{\gamma(\boldsymbol{x})}\widetilde{\boldsymbol{x}}$ where

$$\widetilde{\boldsymbol{x}} \triangleq [\,\boldsymbol{x}\,;\,1\,] \in \mathbb{R}^{n+1}\ ,\ A_c \in \mathbb{R}^{n\times(n+1)}\ \forall c \in \{1,\ldots,N_\mathcal{P}\}\,. \tag{7}$$

Definition 2. f is called CPA if it is continuous and PA.

Fact 1. If f is CPA *w.r.t.* $\mathcal{P}$ it is CPA w.r.t. any $\mathcal{P}' \succ \mathcal{P}$.

A *vector field* $\boldsymbol{v}$ (on Ω) is an $\Omega \to \mathbb{R}^n$ map viewed as the mapping of points to vectors; *i.e.*, if $\boldsymbol{x} \in \Omega$ then $\boldsymbol{v}(\boldsymbol{x})$ is viewed as an n-dimensional "arrow". The terms *velocity field* and *vector field* will be used interchangeably.

Definition 3. A *vector-field space* is a *linear* space whose elements are vector fields.

Let $\mathcal{V}'_{\Omega,\mathcal{P}}$ and $\mathcal{V}_{\Omega,\mathcal{P}}$ be the spaces of PA and CPA velocity fields on Ω w.r.t. $\mathcal{P}$. Note $\mathcal{V}_{\Omega,\mathcal{P}} \subset \mathcal{V}'_{\Omega,\mathcal{P}}$. We will often write $\mathcal{V}'$ and $\mathcal{V}$, suppressing the dependencies on Ω and $\mathcal{P}$.

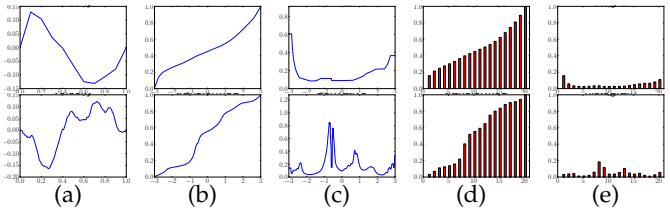
Lemma 1. *$\mathcal{V}'$ and $\mathcal{V}$ are linear spaces, $D \triangleq \dim(\mathcal{V}') = (n^2+n)\times N_\mathcal{P}$, $d \triangleq \dim(\mathcal{V}) \le nN_v$ with equality if and only if $\mathcal{P}$ is of type I, and $d \le D$ with equality if and only if $N_\mathcal{P} = 1$.*

The values of D and d for the tessellations in Fig. 2 appear in Table 1. A generic element of $\mathcal{V}'$ is denoted by $\boldsymbol{v}_{\boldsymbol{A}}$ where $\boldsymbol{A} \triangleq (A_1,\ldots,A_{N_\mathcal{P}})$ consists of its associated matrices (see Eqn. (7)). If $A \in \mathbb{R}^{n\times(n+1)}$, then $\text{vec}(A) \in \mathbb{R}^{n^2+n}$ is its *row-by-row* flattening to a column vector. Likewise, $\mathbf{vec}(\boldsymbol{A}) \triangleq [\,(\text{vec}(A_1))^T\ \ldots\ (\text{vec}(A_{N_\mathcal{P}}))^T\,]^T \in \mathbb{R}^D$. Both $\text{vec}(\cdot)$ and $\mathbf{vec}(\cdot)$ are linear bijections. An inner product on $\mathcal{V}'$ is defined by $\langle \boldsymbol{v}_{\boldsymbol{A}_1}, \boldsymbol{v}_{\boldsymbol{A}_2}\rangle = \mathbf{vec}(\boldsymbol{A}_1)^T\mathbf{vec}(\boldsymbol{A}_2)$.

We now explain how to build *an* orthonormal basis for $\mathcal{V}$, how its elements are parametrized by $\boldsymbol{\theta} \in \mathbb{R}^d$, and how PA velocity fields are projected onto $\mathcal{V}$.

Lemma 2. *An element of $\mathcal{V}$ is any $\boldsymbol{v}_{\boldsymbol{A}} \in \mathcal{V}'$ such that $\mathbf{vec}(\boldsymbol{A})$ satisfies a linear system of constraints, denoted by $L\,\mathbf{vec}(\boldsymbol{A}) = \boldsymbol{0}$.*

Proof. For concreteness, we prove it for $n = 2$ and type-I tessellations, the other cases being similar. Let $\boldsymbol{v}_{\boldsymbol{A}} \in \mathcal{V}'$. While $\boldsymbol{v}_{\boldsymbol{A}}$ is continuous on every cell, it is (usually) discontinuous on cell boundaries. Consider two adjacent cells, U_i and U_j. Let $\boldsymbol{x}_a$ and $\boldsymbol{x}_b$ be their 2 shared vertices and let A_i and A_j denote the corresponding 2×3 matrices. Continuity of $\boldsymbol{v}_{\boldsymbol{A}}$ at $\boldsymbol{x}_a$ implies 2 (more generally, n) linear constraints on A_i and



(a) (b) (c) (d) (e)

Fig. 6: CDF/histogram representation. 1st row: $N_{\mathcal{P}} = 10$. 2nd row: $N_{\mathcal{P}} = 100$. F_0, not shown, is a CDF of a uniform distribution on $J = [-3, 3]$. (a) $\boldsymbol{v}^{\boldsymbol{\theta}} \in \mathcal{V}^{\partial}([0,1], \mathcal{P})$. (b) $F = T^{\boldsymbol{\theta}} \circ F_0$. (c) $\frac{d}{dx}F$. Note it is *not* piecewise constant. H_0, not shown, is a *cumsum* of a 20-bin uniform histogram. (e) $H = T^{\boldsymbol{\theta}} \circ H_0$, a *cumsum* of a new histogram h (f) h.

A_j. Similarly, continuity at $\boldsymbol{x}_b$ implies another constraint pair. Thus, the continuity at both $\boldsymbol{x}_a$ and $\boldsymbol{x}_b$ implies the following 4 linear constraints:

$$\underset{4\times 12}{\begin{bmatrix} \widetilde{\boldsymbol{x}}_a^T & \boldsymbol{0}_{1\times 3} & -\widetilde{\boldsymbol{x}}_a^T & \boldsymbol{0}_{1\times 3} \\ \boldsymbol{0}_{1\times 3} & \widetilde{\boldsymbol{x}}_a^T & \boldsymbol{0}_{1\times 3} & -\widetilde{\boldsymbol{x}}_a^T \\ \widetilde{\boldsymbol{x}}_b^T & \boldsymbol{0}_{1\times 3} & -\widetilde{\boldsymbol{x}}_b^T & \boldsymbol{0}_{1\times 3} \\ \boldsymbol{0}_{1\times 3} & \widetilde{\boldsymbol{x}}_b^T & \boldsymbol{0}_{1\times 3} & -\widetilde{\boldsymbol{x}}_b^T \end{bmatrix}} \underset{12\times 1}{\begin{bmatrix} \mathrm{vec}(A_i) \\ \mathrm{vec}(A_j) \end{bmatrix}} = \boldsymbol{0}_{4\times 1}\,. \tag{8}$$

Continuity at $\boldsymbol{x}_a$ and $\boldsymbol{x}_b$ implies continuity throughout their join, *i.e.*, $A_i\widetilde{\boldsymbol{x}}_a = A_j\widetilde{\boldsymbol{x}}_a$ and $A_i\widetilde{\boldsymbol{x}}_b = A_j\widetilde{\boldsymbol{x}}_b$ imply $(\forall \lambda \in [0,1])$

$$A_i(\lambda\widetilde{\boldsymbol{x}}_a + (1-\lambda)\widetilde{\boldsymbol{x}}_b) = A_j(\lambda\widetilde{\boldsymbol{x}}_a + (1-\lambda)\widetilde{\boldsymbol{x}}_b)\,. \tag{9}$$

Any $\boldsymbol{v}_{\boldsymbol{A}}$ whose A_i and A_j satisfy Eqn. (9) is thus continuous on $U_i \cup U_j$. Similar constraints for other pairs of adjacent cells can be stacked together in an analogous equation:

$$\underset{4N_e\times 6N_{\mathcal{P}}}{L}\ \underset{6N_{\mathcal{P}}\times 1}{\mathbf{vec}(\boldsymbol{A})} = \boldsymbol{0}_{4N_e\times 1} \tag{10}$$

where N_e is the number of shared line segments in $\mathcal{P}$ and L is the constraint matrix. Any $\boldsymbol{v}_{\boldsymbol{A}}$ whose $\boldsymbol{A}$ satisfies Eqn. (10) is thus everywhere continuous. We conclude that the null space of L, denoted by $\mathrm{null}(L)$, coincides with $\mathcal{V}$. □

Let the columns of $\boldsymbol{B} = [\,\boldsymbol{B}_1 \ldots \boldsymbol{B}_d\,] \in \mathbb{R}^{D\times d}$ denote the particular orthonormal basis of $\mathrm{null}(L)$ which is obtained via SVD of L. Let $\boldsymbol{\theta} = [\,\theta_1 \ldots \theta_d\,]^T \in \mathbb{R}^d$. If $\mathbf{vec}(\boldsymbol{A}) = \boldsymbol{B}\boldsymbol{\theta}$ then $\boldsymbol{v}_{\boldsymbol{A}} : \Omega \to \mathbb{R}^n$ is CPA. Particularly, for every $j \in \{1,\ldots,d\}$, $\boldsymbol{v}_{\mathbf{vec}^{-1}(\boldsymbol{B}_j)} : \Omega \to \mathbb{R}^n$ is CPA and $\{\boldsymbol{v}_{\mathbf{vec}^{-1}(\boldsymbol{B}_j)}\}_{j=1}^{d}$ is *a* basis for $\mathcal{V}$. For a visualization of this basis, see **Sup. Mat.** Regardless of whether $\boldsymbol{v}_{\boldsymbol{A}} \in \mathcal{V}'$ is CPA or not, $\boldsymbol{v}_{\mathbf{vec}^{-1}(\boldsymbol{B}\boldsymbol{B}^T\mathbf{vec}(\boldsymbol{A}))}$, its projection on $\mathcal{V}$, is CPA. When earlier we denoted a generic element of $\mathcal{V}$ by $\boldsymbol{v}^{\boldsymbol{\theta}}$ we meant that $\boldsymbol{\theta}$ stands for the coefficients w.r.t. $\{\boldsymbol{v}_{\mathbf{vec}^{-1}(\boldsymbol{B}_j)}\}_{j=1}^{d}$:

$$\boldsymbol{v}^{\boldsymbol{\theta}}(\boldsymbol{x}) = A_{\gamma(\boldsymbol{x}),\boldsymbol{\theta}}\widetilde{\boldsymbol{x}} = \sum\nolimits_{j=1}^{d} \theta_j \boldsymbol{v}_{\mathbf{vec}^{-1}(\boldsymbol{B}_j)}\widetilde{\boldsymbol{x}} \tag{11}$$

where $\boldsymbol{A}_{\boldsymbol{\theta}} \triangleq (A_{1,\boldsymbol{\theta}}, \ldots, A_{N_{\mathcal{P}},\boldsymbol{\theta}}) \triangleq \mathbf{vec}^{-1}\left(\sum_{j=1}^{d}\theta_j\boldsymbol{B}_j\right)$. This basis is orthonormal w.r.t. the inner product

$$\langle\cdot,\cdot\rangle_{\boldsymbol{B}} : \mathcal{V}\times\mathcal{V} \to \mathbb{R}\,,\ (\boldsymbol{v}^{\boldsymbol{\theta}_1}, \boldsymbol{v}^{\boldsymbol{\theta}_2}) \mapsto \boldsymbol{\theta}_1^T\boldsymbol{\theta}_2\,. \tag{12}$$

We refer to it as a *global* basis since each $\boldsymbol{v}_{\mathbf{vec}^{-1}(\boldsymbol{B}_j)}$ impacts $\boldsymbol{v}^{\boldsymbol{\theta}}$ on the entirety of Ω. We will soon describe another orthonormal basis which is more local in nature.

Let $\boldsymbol{v}^{\boldsymbol{\theta}}_{\mathrm{vert}} \triangleq \{\boldsymbol{v}^{\boldsymbol{\theta}}(\boldsymbol{\xi})\}_{\boldsymbol{\xi}\in\mathcal{P}_{\mathrm{vert}}}$ denote the (ordered) N_v-tuple of n-dimensional values $\boldsymbol{v}^{\boldsymbol{\theta}}$ takes at the N_v points in $\mathcal{P}_{\mathrm{vert}}$.

Input: N_{pts} pairs of Ω-valued points: $\{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^{N_{\mathrm{pts}}}$; ε; $k_{\max}$

for $i \in \{1,\ldots,N_{\mathrm{pts}}\}$ **do in parallel**
  $\boldsymbol{x}_i^{[0]} \leftarrow \boldsymbol{x}_i$
for $k \in \{1,\ldots,k_{\max}\}$ **do**
  $\mathrm{obs}^k \leftarrow \{(\boldsymbol{x}_i^{k-1}, \boldsymbol{y}_i)\}_{i=1}^{N_{\mathrm{pts}}}$ `// observations`
  $\boldsymbol{\theta}^k \leftarrow \arg\max_{\boldsymbol{\theta}} p(\boldsymbol{\theta}^k) L_{\boldsymbol{v}}(\boldsymbol{v}^{\boldsymbol{\theta}}; \mathrm{obs}^k)$ where
   $L_{\boldsymbol{v}}(\boldsymbol{v}^{\boldsymbol{\theta}^k}; \mathrm{obs}^k) \propto e^{-\frac{1}{\sigma^2}\sum_{i=1}^{N_{\mathrm{pts}}} \|\boldsymbol{x}_i^{k-1} + \boldsymbol{v}^{\boldsymbol{\theta}}(\boldsymbol{x}_i^{k-1}) - \boldsymbol{y}_i\|^2}$
   `// closed-form optimization; see text`
  $\alpha_k \leftarrow \arg\max_{\alpha} p(\alpha\boldsymbol{\theta}^k) L_T(T^{\alpha\boldsymbol{\theta}^k}; \mathrm{obs}^k)$ where
   $L_T(T^{\alpha\boldsymbol{\theta}^k}; \mathrm{obs}^k) \propto e^{-\frac{1}{\sigma^2}\sum_{i=1}^{N_{\mathrm{pts}}} \|T^{\alpha\boldsymbol{\theta}^k}(\boldsymbol{x}_i^{k-1}) - \boldsymbol{y}_i\|^2}$
   `/* the 1D optimization can be done by,`
   `e.g., a line search; `$\{T^{\alpha\boldsymbol{\theta}^k}(\boldsymbol{x}_i^{k-1})\}_{i=1}^{N_{\mathrm{pts}}}$
   `is computed by Algorithm 1 */`
  if $\|\boldsymbol{\theta}^k\alpha_k\| < \varepsilon$ **then break**;
Output: $\widehat{T} \triangleq (T^{\alpha_k\boldsymbol{\theta}^k} \circ \cdots \circ T^{\alpha_1\boldsymbol{\theta}^{[1]}})$
`//` $\widehat{T}$ `is a concatenation of points in` M

Algorithm 2: A greedy algorithm for (landmark-based) inference over a transformation obtained by integrating a non-stationary field where the field is piecewise-constant w.r.t. time. See text for details.

Lemma 3. *Let $L_{\boldsymbol{\theta}\mapsto\boldsymbol{v}^{\boldsymbol{\theta}}_{\mathrm{vert}}}$ denote the map that sends $\boldsymbol{\theta}$ to $\boldsymbol{v}^{\boldsymbol{\theta}}_{\mathrm{vert}}$. $L_{\boldsymbol{\theta}\mapsto\boldsymbol{v}^{\boldsymbol{\theta}}_{\mathrm{vert}}}$ is a linear injection and its associated $d\times(nN_v)$ matrix is given in closed form. If $\mathcal{P}$ is of type I, the map is invertible. In which case, we denote its inverse by $L_{\boldsymbol{v}^{\boldsymbol{\theta}}_{\mathrm{vert}}\mapsto\boldsymbol{\theta}}$.*

Let $\partial\Omega$ be the border of Ω and $\boldsymbol{n} : \partial\Omega \to S^{n-1}$ be the unit normal to it. We now discuss linear subspaces of $\mathcal{V}$.

Lemma 4. *Optional constraints such as $\langle \boldsymbol{v}^{\boldsymbol{\theta}}(\boldsymbol{x}), \boldsymbol{n}(\boldsymbol{x})\rangle = 0$ on $\partial\Omega$ and/or $\{\mathrm{tr}(A_c) = 0\}_{c=1}^{N_{\mathcal{P}}}$, are linear and can thus extend L (from Lemma 2) to have more rows. The null space of the extended L is a linear subspace (whose dimension is denoted by $d' < d$) of the null space of the original L. Reusing the symbol $\boldsymbol{B} = [\,\boldsymbol{B}_1 \ldots \boldsymbol{B}_{d'}\,] \in \mathbb{R}^{D\times d'}$ to denote the particular orthonormal basis of this new null space which is obtained via SVD of the extended L, we get a d'-dimensional basis of CPA velocity fields, $\{\boldsymbol{v}_{\mathbf{vec}^{-1}(\boldsymbol{B}_j)}\}_{j=1}^{d'}$, whose elements satisfy the new constraint(s).*

The corresponding linear subspaces of $\mathcal{V}$ are denoted $\mathcal{V}^{\partial}$ and $\mathcal{V}^{\mathrm{tr}}$. We also define their intersection, $\mathcal{V}^{\partial,\mathrm{tr}} \triangleq \mathcal{V}^{\partial} \cap \mathcal{V}^{\mathrm{tr}}$. We reuse the symbol $\boldsymbol{v}^{\boldsymbol{\theta}}$ to denote an element of any of these subsets, with the understanding that then $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ for some $d' < d$. We now define, similarly to Eqn. (5), subsets of M:

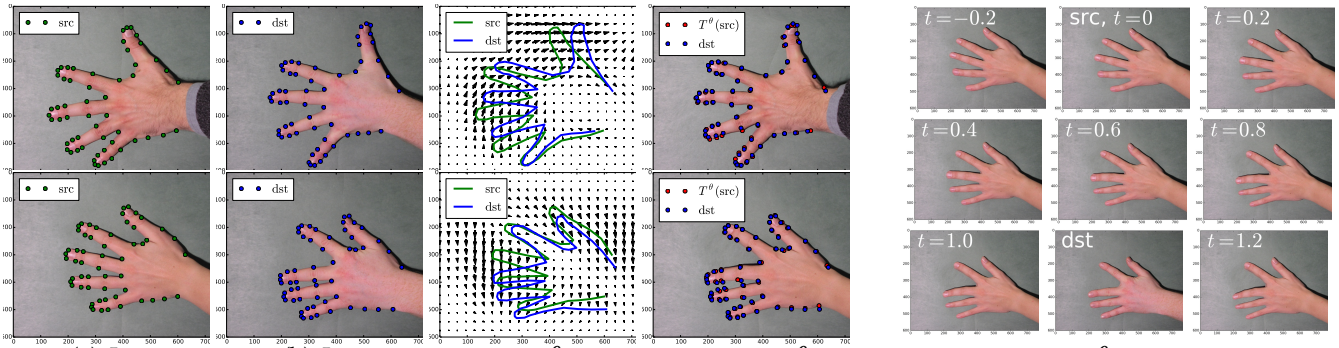
$$M^{\partial} \triangleq \{\exp(\boldsymbol{v}^{\boldsymbol{\theta}}) : \boldsymbol{v}^{\boldsymbol{\theta}} \in \mathcal{V}^{\partial}\} \subset M\,; \tag{13}$$

$$M^{\mathrm{vp}} \triangleq \{\exp(\boldsymbol{v}^{\boldsymbol{\theta}}) : \boldsymbol{v}^{\boldsymbol{\theta}} \in \mathcal{V}^{\mathrm{tr}}\} \subset M\,; \tag{14}$$

$$M^{\partial,\mathrm{vp}} \triangleq \{\exp(\boldsymbol{v}^{\boldsymbol{\theta}}) : \boldsymbol{v}^{\boldsymbol{\theta}} \in \mathcal{V}^{\partial,\mathrm{tr}}\} \subset M\,. \tag{15}$$

If $\mathcal{P}$ is of type I (so $d = nN_v$) then, by viewing $\boldsymbol{v}^{\boldsymbol{\theta}}_{\mathrm{vert}}$ as a point in $\mathbb{R}^d$, the standard basis for $\mathbb{R}^d$ defines another basis for $\mathcal{V}$ (there is a linear bijection between the $(n+1)$-tuple of the n-dimensional velocities at the vertices of a type-I U_c and the $n\times(n+1)$ matrix A_c). Naturally, we refer to this vertex-based basis as a *localized* basis of $\mathcal{V}$. It is orthonormal w.r.t. the following inner product, obtained by summing the dot products between pairs of velocities at the vertices:

$$\langle\cdot,\cdot\rangle_{\mathrm{vert}} : (\boldsymbol{v}^{\boldsymbol{\theta}_1}, \boldsymbol{v}^{\boldsymbol{\theta}_2}) \mapsto \sum\nolimits_{\boldsymbol{\xi}:\boldsymbol{\xi}\in\mathcal{P}_{\mathrm{vert}}} \boldsymbol{v}^{\boldsymbol{\theta}_1}(\boldsymbol{\xi}) \cdot \boldsymbol{v}^{\boldsymbol{\theta}_2}(\boldsymbol{\xi})\,. \tag{16}$$



(a) I_{src} (b) I_{dst} (c) $\boldsymbol{v}^{\boldsymbol{\theta}}$ (d) $I_{\text{src}} \circ T^{\boldsymbol{\theta}}$ (e) $(I_{\text{src}} \circ \boldsymbol{\phi}^{\boldsymbol{\theta}})(\cdot; t)$ and I_{dst}

Fig. 7: Landmark-based warping. One example in each row. (a) I_{src} (and src landmarks). (b) I_{src} (and src/dst landmarks). (c) The inferred $\boldsymbol{v}^{\boldsymbol{\theta}}$. (d) A new image is synthesized by warping I_{src} using $T^{\boldsymbol{\theta}}$ (and dst & $T^{\boldsymbol{\theta}}(\text{src})$ landmarks). (d) Animation.

For $\mathcal{V}^{\partial}$, we can build a similar (d'-dimensional) localized basis; however, for $\mathcal{V}^{\text{tr}}$ (and $\mathcal{V}^{\partial,\text{tr}}$) this is not possible. We conclude Sec. 4.2 with the following fact.

Fact 2. $\langle \cdot, \cdot \rangle_{\boldsymbol{B}}$ and $\boldsymbol{B}$ *vary with translations of* Ω while $\langle \cdot, \cdot \rangle_{\text{vert}}$ and the localized basis are invariant to them.

4.3 Smoothness Priors on CPA Velocity Fields

Adopting a Bayesian approach, we now present two complementary methods to construct priors on CPA fields.

I: Project a prior from $\mathcal{V}'$ onto $\mathcal{V}$. On $\mathcal{V}'$, it is easy to build a smoothness prior that also penalizes large values; *e.g.*, we use a zero-mean Gaussian with a $D \times D$ covariance, $\boldsymbol{\Sigma}_{\text{PA}}$, whose correlations decay with inter-cell distances, and write $\mathbf{vec}(\boldsymbol{A}) \sim \mathcal{N}(\mathbf{0}_{D\times 1}, \boldsymbol{\Sigma}_{\text{PA}})$. Next, we use the $\mathcal{V}' \to \mathcal{V}$ projection, $\boldsymbol{A} \mapsto \boldsymbol{\theta} = \boldsymbol{B}^T \mathbf{vec}(\boldsymbol{A})$, to induce a prior on $\mathcal{V}$:

$$p(\boldsymbol{\theta}) = \mathcal{N}(\mathbf{0}_{d\times 1}, \boldsymbol{\Sigma}_{\text{CPA}}) \tag{17}$$

where $\boldsymbol{\Sigma}_{\text{CPA}} = \boldsymbol{B}^T \boldsymbol{\Sigma}_{\text{PA}} \boldsymbol{B}$. See Fig. 4 for samples from $p(\boldsymbol{\theta})$. By Fact 2, $p(\boldsymbol{\theta})$ depends on the origin of Ω.

II: Let a prior on $\boldsymbol{v}^{\boldsymbol{\theta}}_{\text{vert}}$ induce a prior on $\boldsymbol{v}^{\boldsymbol{\theta}} \in \mathcal{V}$. If the prior on $\boldsymbol{v}^{\boldsymbol{\theta}}_{\text{vert}}$ is invariant to the origin of Ω then so is the resulting $p(\boldsymbol{\theta} = L_{\boldsymbol{v}^{\boldsymbol{\theta}}_{\text{vert}} \mapsto \boldsymbol{\theta}} \boldsymbol{v}^{\boldsymbol{\theta}}_{\text{vert}})$; *e.g.*, such is the case for either a zero-mean Gaussian whose covariance is fully determined by *inter-vertex* distances, or for an MRF of the form $\prod_{i,j:\boldsymbol{\xi}_i \overset{\text{nbrs}}{\sim} \boldsymbol{\xi}_j} \psi(\boldsymbol{v}^{\boldsymbol{\theta}}(\boldsymbol{\xi}_i), \boldsymbol{v}^{\boldsymbol{\theta}}(\boldsymbol{\xi}_j))$ where ψ is some pairwise potential. Method II does not require the computation of $\boldsymbol{B}$. Unlike method I, however, it is applicable only if $\mathcal{P}$ is of type I *and* volume preservation is not imposed. Another advantage of method I over method II (or, for that matters, over most regularizers/priors people often place on velocity fields in the context of diffeomorphisms) is that by defining the prior in terms of the A_c's (as opposed to the $\boldsymbol{v}^{\boldsymbol{\theta}}_{\text{vert}}$ values they generate), *we avoid introducing fictitious variance*. For example, if $\boldsymbol{v}^{\boldsymbol{\theta}}$ is purely affine then the A_c's are all the same. Thus, w.r.t. the prior defined using the global basis (*i.e.*, method I), $\boldsymbol{v}^{\boldsymbol{\theta}}$ is considered very smooth (in the machine-learning sense, not calculus one). However, w.r.t. the localized basis (*i.e.*, method II), $\boldsymbol{v}^{\boldsymbol{\theta}}$ is less smooth since the values of $\boldsymbol{v}^{\boldsymbol{\theta}}_{\text{vert}}$ vary with the vertex location.

4.4 CPA-Based Transformations

Recall from Sec. 3 that, via Eqn. (2), $\boldsymbol{v}^{\boldsymbol{\theta}} \in \mathcal{V}$ implies $T^{\boldsymbol{\theta}} \in M$, a relation denoted $T^{\boldsymbol{\theta}} = \exp(\boldsymbol{v}^{\boldsymbol{\theta}})$ (see Eqn. (5)). The detail alluded to in Sec. 3 is that for Eqn. (2) to be well defined, $\boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, \tau)$ must always be in Ω, the domain of $\boldsymbol{v}^{\boldsymbol{\theta}}$. This trivially holds if $\Omega = \mathbb{R}^n$. It also holds if $\Omega \subsetneq \mathbb{R}^n$ *and* $\boldsymbol{v}^{\boldsymbol{\theta}} \in \mathcal{V}^{\partial}$. If $\Omega \subsetneq \mathbb{R}^n$ and $\boldsymbol{v}^{\boldsymbol{\theta}} \in \mathcal{V} \setminus \mathcal{V}^{\partial}$, it might not. Also, some of the results below require both the continuity and PA properties of $\boldsymbol{v}^{\boldsymbol{\theta}}$. This brings us to the following lemma.

Lemma 5. *If $\Omega \subsetneq \mathbb{R}^n$, we can extend $\mathcal{P}$ and constrain the velocity fields such that we will obtain a linear subspace of $\mathcal{V}$ whose elements extend to CPA fields on the entirety of $\mathbb{R}^n$. In which case, we also redefine Ω to be equal to (the whole of) $\mathbb{R}^n$.*

Henceforth, if $\Omega \subsetneq \mathbb{R}^n$ *and* we use $\mathcal{V}$ (or $\mathcal{V}^{\text{tr}}$) and not $\mathcal{V}^{\partial}$ (or $\mathcal{V}^{\partial,\text{tr}}$), we will assume the procedure from Lemma 5 has been applied. Thus, Eqn. (2) is well defined, $T^{\boldsymbol{\theta}}$ is always an $\Omega \to \Omega$ map and $\boldsymbol{v}^{\boldsymbol{\theta}}$ is indeed CPA. Either way, $\boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, 0) = \boldsymbol{x}$ for every $\boldsymbol{x} \in \Omega$ while $\{\boldsymbol{x} : \boldsymbol{v}^{\boldsymbol{\theta}}(\boldsymbol{x}) = 0\}$ are *fixed points* of $T^{\boldsymbol{\theta}}$.

Fact 3. If $A \in \mathbb{R}^{n\times(n+1)}$, let $\widetilde{A} = \begin{bmatrix} A \\ \mathbf{0}_{1\times(n+1)} \end{bmatrix} \in \mathbb{R}^{(n+1)\times(n+1)}$. The last row of $\mathbf{expm}(\widetilde{A})$ (where $\mathbf{expm}$ is the matrix exponential) is $[\,\mathbf{0}_{1\times n}\ 1\,]$. Also, $\det \mathbf{expm}(\widetilde{A}) > 0$.

If $n = 1$ (so $\widetilde{A}$ is 2 by 2) or if A has a special structure (*e.g.*, if $A_{1:n,1:n+1}$ is either diagonalizable, idempotent, or a 3-by-3 skew-symmetric matrix), then $\mathbf{expm}(\widetilde{A})$ has a closed form. Otherwise, since n is usually small and due to the structure of $\widetilde{A}$, a generic $\mathbf{expm}$ routine typically approximates it well. Let $t \in \mathbb{R}$ and let $\psi^t_{\boldsymbol{\theta},c} : \Omega \to \mathbb{R}^n$ be the solution to an ODE with an $\mathbb{R}^n \to \mathbb{R}^n$ *affine* velocity field, $\boldsymbol{\xi} \mapsto A_{c,\boldsymbol{\theta}}\widetilde{\boldsymbol{\xi}}$, and an initial condition, $\boldsymbol{x}$:

$$\begin{bmatrix} \psi^t_{\boldsymbol{\theta},c}(\boldsymbol{x}) \\ 1 \end{bmatrix} \triangleq T_{A_{c,\boldsymbol{\theta}},t}\widetilde{\boldsymbol{x}}\,, \quad T_{A_{c,\boldsymbol{\theta}},t} \triangleq \mathbf{expm}(t\widetilde{A_{c,\boldsymbol{\theta}}})\,. \tag{18}$$

We note that the solution to Eqn. (2) *is the composition of a finite number, denoted by m, of such solutions:*

$$\boxed{\boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, t) = (\psi^{t_m}_{\boldsymbol{\theta},c_m} \circ \ldots \circ \psi^{t_2}_{\boldsymbol{\theta},c_2} \circ \psi^{t_1}_{\boldsymbol{\theta},c_1})(\boldsymbol{x})}\,. \tag{19}$$

As mentioned earlier, $T^{\boldsymbol{\theta}}$ is defined via $T^{\boldsymbol{\theta}}(\boldsymbol{x}) \triangleq \boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, 1)$. The compact form of Eqn. (19) hides a difficulty: the number of the trajectory segments, m, their durations, $\{t_i\}_{i=1}^m$, and the indices of the cells involved, $\{c_i\}_{i=1}^m$ (where a cell may appear more than once), *all depend on* $\boldsymbol{x}$. Except the index of the first cell, $c_1 = \gamma(\boldsymbol{x})$, they also depend on $\boldsymbol{\theta}$ and t. Thus, a more precise and cumbersome notation would be:

$$(\psi^{t_{m_{\boldsymbol{x},\boldsymbol{\theta},t}}(\boldsymbol{x},\boldsymbol{\theta},t)}_{\boldsymbol{\theta},c_{m_{\boldsymbol{x},\boldsymbol{\theta},t}}(\boldsymbol{x},\boldsymbol{\theta},t)} \circ \ldots \circ \psi^{t_2(\boldsymbol{x},\boldsymbol{\theta},t)}_{\boldsymbol{\theta},c_2(\boldsymbol{x},\boldsymbol{\theta},t)} \circ \psi^{t_1(\boldsymbol{x},\boldsymbol{\theta},t)}_{\boldsymbol{\theta},c_1(\boldsymbol{x})})(\boldsymbol{x})\,.$$

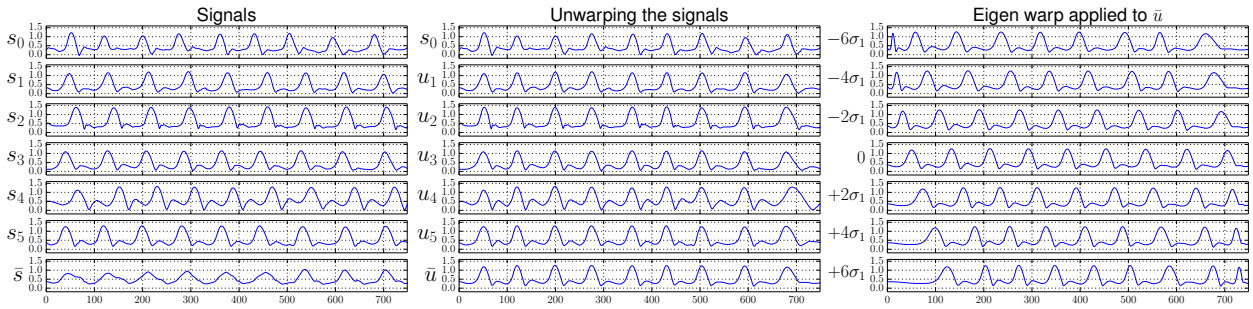


Fig. 8: Nonlinear time warping in real MoCap Data. Left: a reference signal, s_0, 5 other signals, $\{s_i\}_{i=1}^5$, and their mean, $\bar{s} = \frac{1}{5}\sum_{i=1}^5 s_i$. The mean is smeared since $\{s_i\}_{i=1}^5$ are misaligned. Center: having inferred the warps, $\{T^{\boldsymbol{\theta}_i}\}_{i=1}^5$, we unwarp the signals, setting $u_i = s_i \circ T^{-\boldsymbol{\theta}_i}$. Note $\{u_i\}_{i=1}^5$ are better aligned, as is evident by the details preserved in their mean, $\bar{u} = \frac{1}{5}\sum_{i=1}^5 u_i$. Right: warping $\bar{u}$ by $T^{\bar{\boldsymbol{\theta}}+j\sigma_1\xi_1}$ where $\bar{\boldsymbol{\theta}} = \frac{1}{5}\sum_{i=1}^5 \boldsymbol{\theta}_i$, $j \in \{-6,-4,-2,0,2,4,6\}$, ξ_1 is the first principal component of $\{\boldsymbol{\theta}_i - \bar{\boldsymbol{\theta}}\}_{i=1}^5$, and σ_1 is the corresponding standard deviation.

Remark 5. While invertible affine matrices form a group, $\boldsymbol{\phi}^{\boldsymbol{\theta}}(\cdot,t) : \Omega \to \Omega$ is *not* affine, exactly because of the $\boldsymbol{x}$-dependencies mentioned above. In fact, since *the above quantities vary with* $\boldsymbol{x}$ *even within a cell*, the continuous $\boldsymbol{x} \mapsto \boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x},t)$ is not PA, hence not CPA.

A map, $T : \Omega \to \Omega$, is a called a (C^1) *diffeomorphism* (on Ω) if T^{-1} exists and both T and T^{-1} are differentiable. Let G be the space of (orientation-preserving) diffeomorphisms on Ω. Let $H \subset G$ be its restriction to those diffeomorphisms that can be obtained, via integration, from uniformly-continuous stationary velocity fields; *i.e.*, $T(\cdot) = \boldsymbol{\phi}(\cdot,1)$ where $\phi(\boldsymbol{x},t) = \boldsymbol{x} + \int_0^t \boldsymbol{v}(\phi(\boldsymbol{x},\tau))\,d\tau$ for a uniformly-continuous $\boldsymbol{v} : \Omega \to \mathbb{R}^n$. Both G and H are $\infty-$dimensional nonlinear spaces.

Theorem 1. *(i) If* $\alpha \in \mathbb{R}$ *then* $\boldsymbol{v}^{\boldsymbol{\theta}} = \boldsymbol{v}^{\alpha\boldsymbol{\theta}}/\alpha$ *and* $\boldsymbol{\phi}^{\boldsymbol{\theta}}(\cdot,t) = \phi^{\alpha\boldsymbol{\theta}}(\cdot,t/\alpha)$. *(ii)* $(T^{\boldsymbol{\theta}})^{-1}$ *exists, is in* M*, and equals to* $T^{-\boldsymbol{\theta}}$. *(iii)* $M \subset G$. *(iv) If* $T^{\boldsymbol{\theta}} \in M^{\mathrm{vp}}$ *then* $T^{\boldsymbol{\theta}}$ *is volume-preserving (hence the superscripted* vp*). (v)* M *is a* d*-dimensional nonlinear space; similar statements hold for* M^{∂}, M^{vp} *and* $M^{\partial,\mathrm{vp}}$*, with their respective values of* d*'. (vi) If* $\mathcal{P}_1 \prec \mathcal{P}_2 \prec \ldots$ *is a tessellation sequence such that eventually all the cells become arbitrarily small, then* $M_{\Omega,\mathcal{P}_k} \xrightarrow{k\to\infty} H$.

Remark 6. Since M is nonlinear, it is easier to work in the linear $\mathcal{V}$ and then, via exp, move from $\mathcal{V}$ to M. This (tangent-space-based) approach is popular in the context of nonlinear differentiable manifolds and, particularly, is used extensively in various spaces of diffeomorphisms (e.g., [27]).

Corollary 1. *If* $n = 1$, $T^{\boldsymbol{\theta}}$ *is increasing. Let* $J \subset \mathbb{R}$ *be an interval. If* $F_0 : J \to \Omega$ *is either non-decreasing, increasing, right-continuous, continuous, differentiable, a diffeomorphism, or a step function (7 non-mutually-exclusive cases), then so is* $F^{\boldsymbol{\theta}} \triangleq T^{\boldsymbol{\theta}} \circ F_0 : J \to \Omega$. *If, in addition,* $\Omega = [0,1]$, $\boldsymbol{v}^{\boldsymbol{\theta}} \in \mathcal{V}^{\partial}$ *and* F_0 *is a CDF, then* $F^{\boldsymbol{\theta}}$ *is a CDF. The (trivial) proof is omitted.*

This gives an unconstrained parametrization of large classes of increasing functions and CDFs/histograms via a finite-dimensional linear space (Figs. 5 and 6). In the CDF case, F_0 need have neither a density nor finite moments.

4.5 Integration Details

Equation (19) justifies and simplifies the proof of Theorem 1, provides an insight into the structure of $t \mapsto \boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x},t)$, and, for $n = 1$, yields a closed-form solution for $\boldsymbol{x} \mapsto T^{\boldsymbol{\theta}}(\boldsymbol{x})$. If $n > 1$, it suggests a practical, *essentially*-exact solution.

Theorem 2. *If* $n = 1$ *then* $m_{\boldsymbol{x},\boldsymbol{\theta},t}$, $\{t_i(\boldsymbol{x};\boldsymbol{\theta},t)\}_{i=1}^{m_{\boldsymbol{x},\boldsymbol{\theta},t}}$, *and* $\{c_i(\boldsymbol{x};\boldsymbol{\theta},t)\}_{i=2}^{m_{\boldsymbol{x},\boldsymbol{\theta},t}}$ *have closed forms. Thus, so does* $T^{\boldsymbol{\theta}}(\boldsymbol{x})$.

Proof. The notation below drops the $\boldsymbol{\theta}$-dependencies. Without loss of generality, let $t > 0$. For $c \in \{c_i(\boldsymbol{x};t)\}_{i=2}^{m_{\boldsymbol{x},t}}$, let $A_c = [a_c, b_c]$ and let $U_c = [x_c^{\min}, x_c^{\max}]$. It can be shown that

$$\psi_c^t(\boldsymbol{x}) = \begin{cases} e^{ta_c}x + \frac{b_c(e^{ta_c}-1)}{a_c} & \text{if } a_c \neq 0 \\ x + tb_c & \text{if } a_c = 0 \end{cases}. \tag{20}$$

Note $c_1 = \gamma(\boldsymbol{x})$ and suppose $\boldsymbol{v}(\boldsymbol{x}) > 0$. Let $t_1(\boldsymbol{x})$ be the hitting time of the right boundary of U_{c_1}:

$$t_1 \triangleq \min\{t : \psi_{c_1}^t(\boldsymbol{x}) = \boldsymbol{x}_{c_1}^{\max}\} \tag{21}$$

where, by convention, the minimum of an empty set is ∞. If $\boldsymbol{v}(\boldsymbol{x}_{c_1}^{\max}) \leq 0$ then $t_1 = \infty$. Otherwise,

$$t_1 = \begin{cases} \frac{1}{a_{c_1}} \log\left(\frac{\boldsymbol{x}_{c_1} + \frac{b_{c_1}}{a_{c_1}}}{\boldsymbol{x} + \frac{b_{c_1}}{a_{c_1}}}\right) & \text{if } a_{c_1} \neq 0 \\ \frac{x_{c_1} - \boldsymbol{x}}{b_{c_1}} & \text{if } a_{c_1} = 0 \end{cases}. \tag{22}$$

If $t_1 > t$, we are done: $\boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x},t) = \psi_{c_1}^t(\boldsymbol{x})$. Otherwise, we entered the next interval to the right, U_{c_1+1} (so $c_2 = c_1 + 1$), and the process (*i.e.*, solving for t_2, but this time with $x_{c_1}^{\max}$ as the new starting point instead of $\boldsymbol{x}$, with $t - t_1$ instead of t, and c_2 instead of c_1) is redone iteratively till convergence (the number of such steps, $m_{\boldsymbol{x},t}$, is finite since t is finite). A loose upper bound on $m_{\boldsymbol{x},t}$ is $N_{\mathcal{P}} - c_1 + 1$. The case where $\boldsymbol{v}(x) < 0$ is handled similarly. Taken together,

$$\boldsymbol{\phi}(\boldsymbol{x};t) = (\psi_{c_m}^{t_m} \circ \ldots \circ \psi_{c_2}^{t_2} \circ \psi_{c_1}^{t_1})(\boldsymbol{x}) \tag{23}$$

where for $2 \leq i \leq m_{\boldsymbol{x},t}$, $c_i = c_{i-1} + \mathrm{sign}(\boldsymbol{v}(\boldsymbol{x}))$, while $m_{\boldsymbol{x},t}$ and $\{t_i\}_{i=1}^{m_{\boldsymbol{x},t}}$ are found as describe above. □

If $n > 1$ then $T^{\boldsymbol{\theta}}(\boldsymbol{x})$ is given *essentially* in closed from in the sense that Eqn. (19) still holds, but there is the mild issue

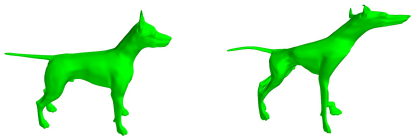


Fig. 9: A 3D example. Left: Source shape (taken from the Tosca Dateset: http://tosca.cs.technion.ac.il) . Right: Result of applying a volume-preserving transformation, sampled from the prior, to the source.

that a generic **expm** is needed (see also the matrix decomposition suggested in [27]) and, much more importantly, to find the quantities from Theorem 2 one needs, for every $\boldsymbol{x}$, to sequentially solve, an $\boldsymbol{x}$-dependent number of problems of the form $\arg\min_{t>0} \gamma([\mathbf{expm}(t\widetilde{A})]_{1:n,1:n+1}\widetilde{\boldsymbol{\xi}}) \neq \gamma(\boldsymbol{\xi})$ where the pair $(\boldsymbol{\xi}, A) \in \Omega \times \mathbb{R}^{n\times(n+1)}$ is (in general) different in each problem. Doing this for $n = 1$ is easy, fast, and accurate. But for $n \geq 2$, partly since $\boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, \cdot)$ *may reenter a cell* (prohibiting, e.g., a binary-search method), this requires tedious bookkeeping, multiple **expm** calls, and multiple invocations of a numerical solver. This yields slow approximated solutions that are also hard to implement in GPU.

Inference over diffeomorphisms (CPAB included) often requires evaluating $T^{\boldsymbol{\theta}}$ for multiple values of $\boldsymbol{\theta}$ (*e.g.*, 10,000), and a high N_{pts}, where N_{pts} is number of points to be transformed (*e.g.*, the number of image pixels). Preferring a more practical alternative, we propose a specialized solver for integrating CPA fields. This solver, summarized in Algorithm 1, alternates between the analytic solution (with neither bookkeeping nor need to solve explicitly for $m_{\boldsymbol{x},\boldsymbol{\theta},1}$, $\{c_i(\boldsymbol{x};\boldsymbol{\theta},1)\}_{i=2}^{m_{\boldsymbol{x},\boldsymbol{\theta}}}$ and $\{t_i(\boldsymbol{x};\boldsymbol{\theta},1)\}_{i=1}^{m_{\boldsymbol{x},\boldsymbol{\theta}}}$) and a generic solver. Thus, the proposed specialized solver is both faster and more accurate than a generic solver since most of the trajectory is solved exactly, while only in small portions of the trajectory does our solver resort to the generic one. Importantly, in Algorithm 1, the required number of **expm** calls is constant, $N_{\mathcal{P}}$, and grows with neither N_{pts} nor t. In fact, since Algorithm 1 is highly accurate, fast, and simple, and since usually $N_{\mathcal{P}} \ll N_{\text{pts}}$, we prefer to use it even when $n = 1$. Lower/upper bounds on the algorithm's complexity are $O(C_1) + O(C_2 \times N_{\text{steps}})$ and $O(C_1) + O(C_2 \times N_{\text{steps}} \times n_{\text{steps}})$ where $C_2 = N_{\text{pts}}/\#\text{cores}$ and $C_1 = \text{expm's cost} \times N_{\mathcal{P}}/\#\text{cores}$. Usually, the $O(C_1)$ term is negligible. $N_{\mathcal{P}}$ and $\boldsymbol{\theta}$ affect inter-cell moves hence which bound is tighter; usually it is the lower one.

Remark 7. Consecutive analytic updates (for the same $\Delta_t\widetilde{A_c}$) amount to taking powers of $\mathbf{expm}(\Delta_t\widetilde{A_c})$. While $(\mathbf{expm}(\Delta_t\widetilde{A_c}))^k = \mathbf{expm}(k\Delta_t\widetilde{A_c})$, on a computer numerical errors accrue and the equality may not hold [66], [67]. This issue is most noticeable when the matrices are large (and dense) and when k is large. Here, however, this issue does not arise in practice since both the matrices (which also have zeros in the last row) and k are small.

In Algorithm 1, γ is called often (though far less often than when using a generic solver throughout). In our implementation, we decided against a lookup-table approach since, once the trajectories start evolving, the argument to γ takes values in the continuum. Besides their simplicity, a key advantage of type-II cells, most notably when compared with type-III cells, is that evaluating γ is easy and fast, especially when $\mathcal{P}$ is also regular (since we can then use rounding and modulo operations). Thus, *e.g.*, we construct our type-I tessellations as a refinement of regular type-II tessellation. If $n = 2$, we split each rectangle into 4 triangles while if $n = 3$ we split each box into 5 tetrahedra. When evaluating γ, we first find the "parent" hyperrectangle and then pick the correct type-II cell among those within that hyperrectangle. For $n > 3$, we avoided implementing type-I cells; rather, in such cases we use only type-II tessellations.

4.6 Adaptive Tessellations

It is possible to use CPAB transformations with a data-adaptive $\mathcal{P}$; *e.g.*, placing more cells in certain regions. The potential expressiveness gain, however, needs to be compared against: 1) the slowdown in evaluating γ and thus in evaluating $\boldsymbol{x} \mapsto T^{\boldsymbol{\theta}}(\boldsymbol{x})$; 2) the need to recompute $\boldsymbol{B}$ (unless one is content with a localized basis and priors obtained via method II). In any case, while for high values of n (say, $n > 3$) this may be a reasonable approach to mitigate the curse of dimensionality, we prefer, to keep computing time low and implementation simple, to use fixed tessellations and handle scarce-data regions via the smoothness prior.

4.7 Computing Derivatives of $T^{\boldsymbol{\theta}}(\boldsymbol{x})$ w.r.t. $\boldsymbol{\theta}$

One of our main motivations was facilitating tractable MCMC inference over latent diffeomorphisms. While we empirically found MCMC methods to be easier to tune and to produce better results than gradient-based optimization, for completeness we provide the details for computing $dT^{\boldsymbol{\theta}}(\boldsymbol{x})/d\boldsymbol{\theta}$. While one can also use finite differences, the derivation below leads to better accuracy and some computational savings (e.g., no additional **expm** calls are needed). Let $\boldsymbol{\theta}_j$ denote the j^{th} component of $\boldsymbol{\theta}$. Define $y(\boldsymbol{x}, t) \triangleq \frac{d}{d\boldsymbol{\theta}_j}\boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, t)$. It can be shown that $y(\boldsymbol{x}, t)$ satisfies

$$\underbrace{y(\boldsymbol{x},t)}_{n\times 1} = \int_0^t \underbrace{B_{c,j}}_{n\times(n+1)} \underbrace{\boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x},\tau)}_{n\times 1} + \underbrace{A_{c,\boldsymbol{\theta}}}_{n\times(n+1)} \underbrace{y(\boldsymbol{x},\tau)}_{n\times 1} d\tau \quad (24)$$

where $c = \gamma(\boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, \tau))$ and $B_{c,j}$ is the c^{th} matrix in $\mathbf{vec}^{-1}(\boldsymbol{B}_j) = (B_{1,j}, \ldots, B_{N_{\mathcal{P}},j})$. Setting $t = 1$, the solution of these d equations (for $j \in \{1, \ldots, d\}$) yields $\frac{dT^{\boldsymbol{\theta}}(\boldsymbol{x})}{d\boldsymbol{\theta}}$. Any generic numerical solver for integral equations can be used to solve these equations, where the required values of $\boldsymbol{\phi}^{\boldsymbol{\theta}}(\boldsymbol{x}, \tau)$ can be computed using Algorithm 1. We omit the details, but our implementation includes this derivative.

4.8 Time-Dependent Fields and Their Inference

This paper focuses on stationary CPA fields but, more generally, $\boldsymbol{\theta}$ may vary with t. In which case, however, Algorithm 1 and Eqn. (19) no longer apply (but note that most of the other advantages of the proposed approach will still apply; e.g., relatively-low dimensionality, compact representation, etc.). While we are unaware of a way to exploit the structure of general non-stationary CPA fields for better integration, we can still apply any generic integration technique to them. That said, one useful case of non-stationary fields is special:

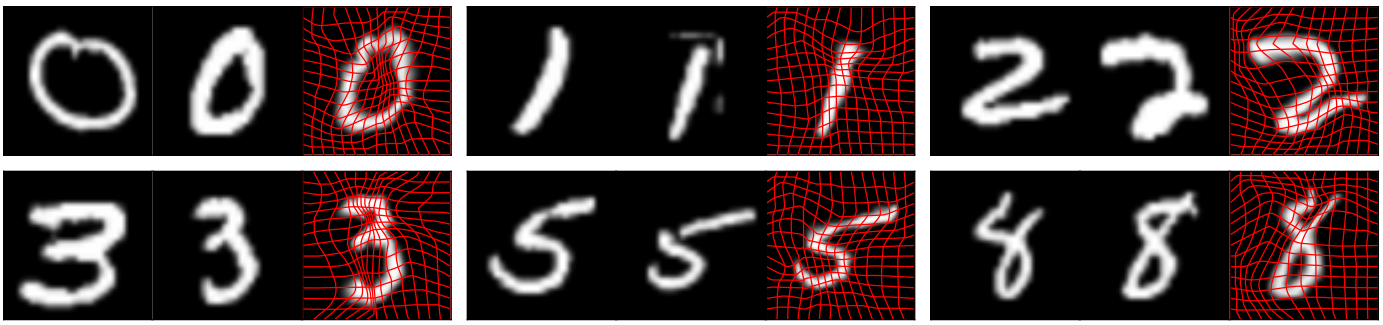


Fig. 10: Example registrations of images from the MNIST dataset

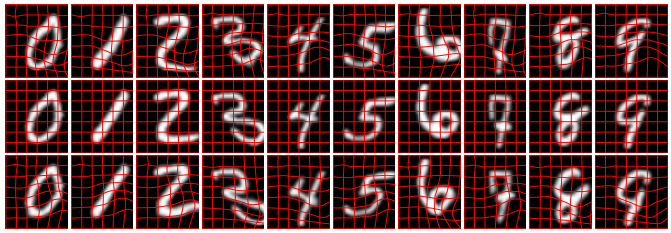


Fig. 11: The 1^{st} eigen wrap of class. *Center row:* the mean (identity) transform. *Top row:* the mean plus 3 standard deviations. *Bottom row:* the mean minus 3 standard deviations.

if $\boldsymbol{\theta}(t)$ is piecewise constant w.r.t. t, then we (trivially) apply Algorithm 1 consecutively, once within each time interval. Here is a usage example. Given N_{pts} pairs of Ω-valued points, $\{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^{N_{\text{pts}}}$, consider an optical-flow-like representation, $\boldsymbol{x} \mapsto \boldsymbol{x} + \boldsymbol{v}^{\boldsymbol{\theta}}(\boldsymbol{x})$, together with the following Gaussian likelihood model,

$$\boldsymbol{y}_i \overset{\text{I.I.D.}}{\sim} \mathcal{N}(\boldsymbol{x}_i + \boldsymbol{v}^{\boldsymbol{\theta}}(\boldsymbol{x}_i), \sigma^2 I_{n\times n}) \tag{25}$$

($\sigma > 0$), and a Gaussian prior for $\boldsymbol{\theta}$ as in, *e.g.*, Eqn. (17). The following $\Omega \times \mathbb{R}^d \to \mathbb{R}^n$ map,

$$(\boldsymbol{x}, \boldsymbol{\theta}) \mapsto \boldsymbol{x} + \boldsymbol{v}^{\boldsymbol{\theta}}(\boldsymbol{x}) \overset{\text{Eqn. (11)}}{=} \boldsymbol{x} + \sum\nolimits_{j=1}^{d} \theta_j \boldsymbol{v}_{\mathbf{vec}^{-1}(\boldsymbol{B}_j)}\widetilde{\boldsymbol{x}}\,, \tag{26}$$

is affine w.r.t. $\boldsymbol{\theta} = [\theta_1, \ldots, \theta_d]^T$. It follows that there is a closed-form MAP solution for $\boldsymbol{\theta}$ (obtained by applying the standard result of computing the conditional mean of one random variable given the other, when both are jointly Gaussian). Let $\widehat{\boldsymbol{\theta}}_{\text{MAP}}$ denote this solution. Importantly, the $\Omega \to \mathbb{R}^n$ map,

$$\boldsymbol{x} \mapsto \boldsymbol{x} + \boldsymbol{v}^{\widehat{\boldsymbol{\theta}}_{\text{MAP}}}(\boldsymbol{x})\,, \tag{27}$$

is (usually) not a diffeomorphism; however, for any real scalar α, the $\Omega \to \Omega$ map,

$$\boldsymbol{x} \mapsto T^{\alpha\widehat{\boldsymbol{\theta}}_{\text{MAP}}}(\boldsymbol{x}) = \exp(\boldsymbol{v}^{\alpha\widehat{\boldsymbol{\theta}}_{\text{MAP}}})(\boldsymbol{x})\,, \tag{28}$$

is a (CPAB) diffeomorphism. This suggests a practical greedy approach for a quick-and-dirty inference over (a transformation obtained by the integral of) a CPA velocity field which is piecewise-constant w.r.t. time. This method, summarized in Algorithm 2, may be adjusted to the case of no-correspondences image registration setting.

4.9 Some Differential-Geometric Aspects

Readers less interested in differential geometry may safely skip this section. As many readers must have recognized, $\mathcal{V}$ is the tangent space at the identity to M; *i.e.*, $\mathcal{V} = T_I M$. Like the transformation space from [27], M is not a Lie group (lack of closure under composition); equivalently, $\mathcal{V}$ is not a Lie algebra (lack of Lie-bracket closure). Note that $\mathcal{V}$ is a finite-dimensional linear subspace of the (∞-dimensional) Lie algebra, $T_I G$ (the Lie group G was defined before Theorem 1), and that $\exp : \mathcal{V} \to M$ is the restriction of the Lie-group exponential map, $\exp : T_I G \to G$, to $\mathcal{V} \subset T_I G$. This approach, of restricting a Lie group exponential map to a linear subspace (of the Lie algebra) lacking a Lie-bracket closure, was found useful, *e.g.*, in the context of Symmetric Positive-Definite (SPD) matrices (SPD is not closed under the standard binary operation of matrix Lie groups; likewise, T_ISPD is not closed under the Lie Bracket.) [39]. The inner product (on $\mathcal{V} = T_I M$) in Eqn. (16) is close in spirit to the oft-used inner product (on $T_I G$):

$$\langle \boldsymbol{v}_1, \boldsymbol{v}_2 \rangle = \int_\Omega \boldsymbol{v}_1(\boldsymbol{\xi}) \cdot \boldsymbol{v}_2(\boldsymbol{\xi})\, d\boldsymbol{\xi} \quad \boldsymbol{v}_1, \boldsymbol{v}_2 \in T_I G\,. \tag{29}$$

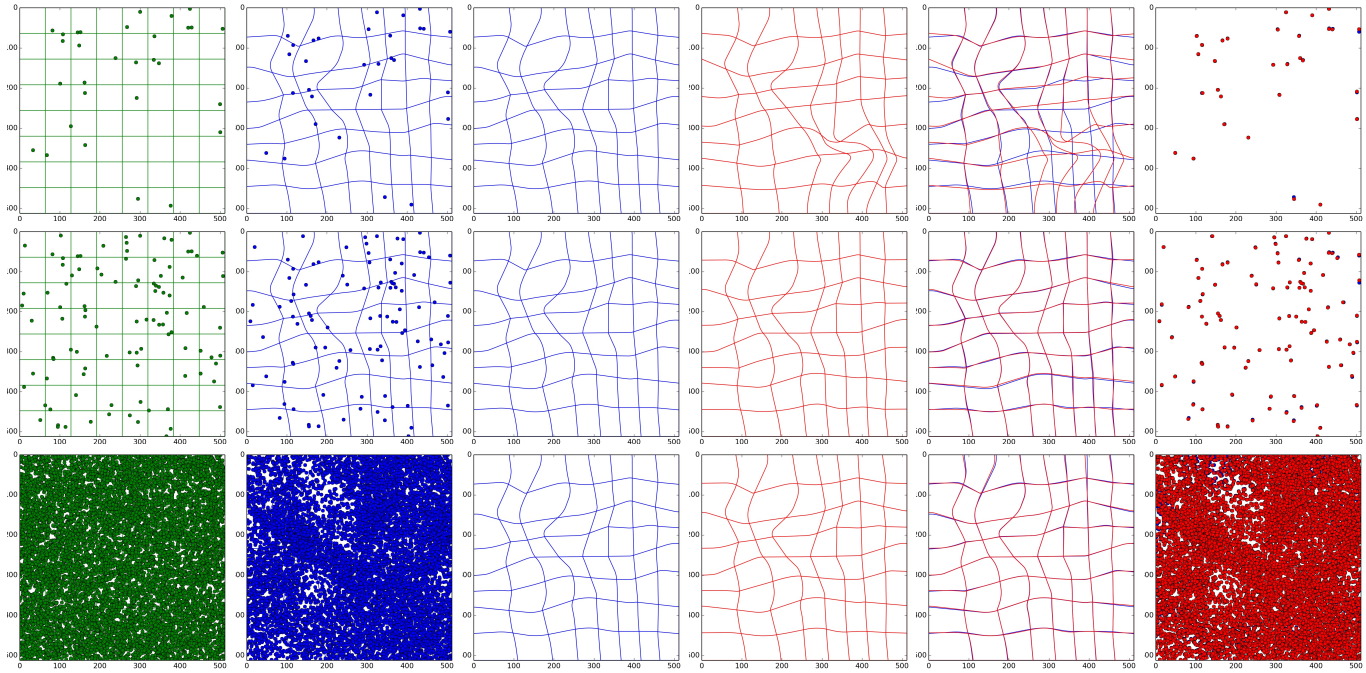
As we pointed out, one of the advantages of Eqn. (12) over Eqn. (16), is related to eliminating fictitious noise. We are unaware of works (including those using Sobelev spaces) suggesting an inner product on $T_I G$ with a similar property. While here we adopt an approach exploiting (the restriction of) the Lie group exponential of $G \supset M$, future work should explore Riemannian geometries of M (with their Riemannian exponential maps) and their relations to those of G. While one can define M-specific Riemannian metrics, note that M may also inherit a Riemannian metric from G.

5 Experiments

Sections 5.1, 5.2 and 5.3 focus on synthesis, Sec. 5.4 pertains to analysis by synthesis, Sec. 5.5 addresses large numbers of landmarks, Sec. 5.6 is related to Algorithm 2, and Sec. 5.7 illustrates representation of CDFs.

5.1 Integration Accuracy and Timings

The two parameters in Algorithm 1, N_{steps} and n_{steps}, imply two step sizes, one large, $\Delta_t = t/N_{\text{steps}}$, used for exact updates, and one small, $\delta_t = \Delta_t / n_{\text{steps}}$, used for the generic-solver subroutine. The reason we can make large (hence



(a) src: grid lines & landmarks (b) dst: grid lines & landmarks (c) dst: grid lines (d) $T^{\boldsymbol{\theta}}$(dst): grid lines (e) dst and $T^{\boldsymbol{\theta}}$(dst): grid lines (f) dst and $T^{\boldsymbol{\theta}}$(dst): landmarks

Fig. 12: Landmark-based Maximum Likelihood (ML) estimates of latent transformations. The dimension of the transformation space in this example is $d = 38$. Rows differ in the number of landmarks, N_{pts}. Top row: $N_{\text{pts}} = 30$. Middle: $N_{\text{pts}} = 100$. Bottom: $N_{\text{pts}} = 10,000$. Colors: green=src, blue=dst, red=$T^{\boldsymbol{\theta}}$(src), where $T^{\boldsymbol{\theta}}$ is the ML transformation, found by Metropolis' algorithm. (a) original grid lines + src landmarks; (b) grid lines deformed by the Ground-Truth (GT) transformation + dst landmarks. (c) grid lines deformed by the GT transformation. (d) grid lines deformed by the ML transformation. (e) grid lines deformed by the GT transformation (blue) and grid lines deformed by the ML transformation (red). (f) dst landmarks (blue) and the src landmarks transformed by the ML transformation (red).

few) steps is that we use an exact analytic update when possible. In our experiments, the generic solver was the modified Euler Method, which requires (i) the ability to evaluate a velocity field at a given location and (ii) specifying the step size and number of steps. By construction, Algorithm 1 is more accurate (due to the analytic updates) and faster (due to the smaller number of steps) than any generic solver used as its subroutine. We also verified this empirically, comparing the algorithm against simply running the generic solver step size δ_t and $N_{\text{steps}} \times n_{\text{steps}}$ steps. For accuracy, in 1D, we compared the integration error (that we can compute this error at all, is by the virtue of our closed-form solution; see Theorem 2) averaged over random CPA velocity fields; *i.e.*, for each value of $N_{\mathcal{P}}$, we drew 100 CPA fields form the prior, $\{\boldsymbol{v}^{\boldsymbol{\theta}_i}\}_{i=1}^{100}$, sampled (uniformly) 1000 points in Ω, $\{x_j\}_{j=1}^{1000}$, and then computed the error:

$$\epsilon = \frac{1}{100}\sum_{i=1}^{100}\frac{1}{1000}\sum_{i=j}^{1000}\left|T^{\boldsymbol{\theta}}_{\text{exact}}(x_j) - T^{\boldsymbol{\theta}}_{\text{solver}}(x_j)\right|. \quad (30)$$

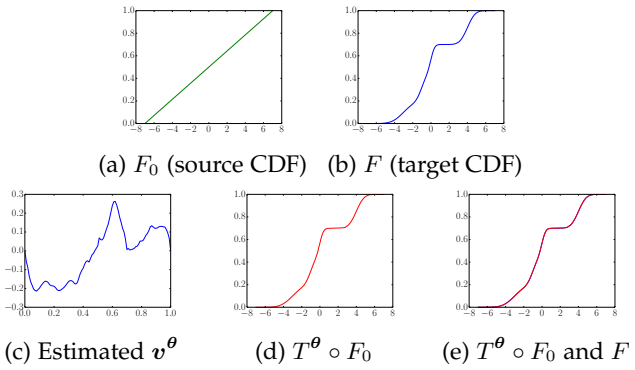
Table 2 shows the relative improvements, $\frac{\epsilon_{\text{generic}} - \epsilon_{\text{algorithm}}}{\epsilon_{\text{generic}}}$, for different values of $N_{\mathcal{P}}$ (the number of intervals). For higher dimensions, where closed form is unavailable, such error analysis is impossible. The results of our timing experiments appear in the **. Sup. Mat.**. Note that our implementation can be further optimized. *e.g.*, use a C++ wrapper instead of a Python one. Moreover, most of the computing time is spent on evaluating γ (γ is continuously-defined, so discretely-defined lookup tables will not do); with some tricks, the number of such calls can be drastically reduced.

5.2 Real-Time Diffeomorphic Image Editing

To highlight the speed at which CPAB transformations are computed, our code includes a GUI where a user chooses velocities at some/all vertices of $\mathcal{P}_4$. Using Lemma 3, the Gaussian priors on $\boldsymbol{v}^{\boldsymbol{\theta}}$, and the fast integration, we compute, in real time, the conditional warp, $\exp(E(\boldsymbol{v}^{\boldsymbol{\theta}}|\text{user's choice}))$. For a demo, please see the **video** in the **Sup. Mat.**

5.3 Expressiveness

While simple, CPAB transformations capture a wide range of diffeomorphisms (obviously, like any other work on diffeomorphisms, we make no claims about capturing non-diffeomorphisms such as occlusions). One way to see this is by inspecting samples from $p(\boldsymbol{\theta})$ (Fig. 4). M also contains what is sometimes called large deformations (an overloaded term: used in some texts for elements of G outside its identity component). One way to show a nominal space captures these is a rectangle-to-horseshoe morphing, achieved easily here (see Sup. Mat.). Expressiveness is also exemplified by the experiments below (Figs. 7 and 8) and the demo (Sec. 5.2). As another example, Fig. 9 shows a volume-preserving transformation sampled from the prior applied to a 3D shape. Here, $T^{\boldsymbol{\theta}} \in M^{\partial,\text{vp}}$, where $\mathcal{P}$ consists of



(a) F_0 (source CDF) (b) F (target CDF)

(c) Estimated $\boldsymbol{v}^{\boldsymbol{\theta}}$ (d) $T^{\boldsymbol{\theta}} \circ F_0$ (e) $T^{\boldsymbol{\theta}} \circ F_0$ and F

Fig. 13: CDF Representation. The source is the CDF of the uniform distribution. The target is the CDF of a 3-component Gaussian mixture. The setting and the inference procedure, where we used a Maximum Likelihood (ML) solution obtained using MCMC, are virtually the same as in the monotonic-regression experiment, except here we also used simulated annealing, gradually reducing the standard deviation of the "noise" to zero. Typical to an ML solution, the inferred $\boldsymbol{v}^{\boldsymbol{\theta}}$ is non-smooth. MAP estimation (not shown) leads to a smoother field, analogously to the results in Fig. 5.

$N_{\mathcal{P}} = 320$ cells (a layout of $4 \times 4 \times 4$ cubes, each divided into 5 tetrahedra) and $N_v = 125$. The dimension, $\dim(M^{\partial,\mathrm{vp}})$, is 20 (and not $375 = 3N_v$) due to the additional constraints.

5.4 Inference: Analysis-by-Synthesis

Real-world signal analysis requires 3 oft-confounded parts: a *representation*; a probabilistic/deterministic *model* (*e.g.*, a likelihood); *inference*. While the first affects the second which in turn affects the third, when judging a new choice for any of these, it is crucial to distinguish between them. For example, as noted in [68], Horn and Schunk's representation+model of optical flow had been regarded highly inaccurate, till modern inference rendered it almost state-of-the-art. Since *the present work is focused on representations*, we avoid advocating a particular likelihood model or inference method, and instead use, in a coarse-to-fine manner (recall that if $\boldsymbol{v}^{\boldsymbol{\theta}}$ is CPA $w.r.t.$ $\mathcal{P}$ it is CPA w.r.t. $\mathcal{P}' \succ \mathcal{P}$), simple choices such as Gaussian likelihood and either conjugate-gradient optimization or the MCMC Metropolis algorithm (where, in the proposal step, we use either global or local moves, utilizing Lemma 3). Without claiming these choices are optimal, these turned out to be effective.

Latent transformations arise in two typical settings: with and without known correspondences. In the first, also called (exact/inexact) diffeomorphic point matching [10], [14], one has a set of landmark pairs, $\{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^{N_{\mathrm{pts}}}$ and seeks $T : \Omega \to \Omega$ such that $\{T(\boldsymbol{x}_i) \approx \boldsymbol{y}_i\}_{i=1}^{N_{\mathrm{pts}}}$. We observe that for $n = 1$, by Corollary 1, our ability to easily solve this problem using CPAB transformations lets us solve a *monotonic regression* problem, as Fig. 5 demonstrates. Particularly, by adding boundary constraints and simulated annealing, we can well fit a CPAB transformation to any reasonable CDF. More generally, but still for $n = 1$, our method can be used to solve statistical/optimization problems over monotonic functions, CDFs

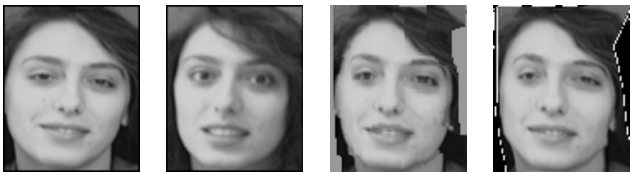


Fig. 14: "Prettifying" the result of a dense-correspondence tool. From left to right: I_{src}; I_{dst}; warping I_{src} according to SIFT-Flow [20]; warping I_{src} using the output of Algorithm 2 where the SIFT-Flow was the input.

and (cumulative sums of) histograms/filters while respecting bin ordering. The case of $n > 1$ appears, *e.g.*, in landmark-based image warping. As shown in Fig. 7, using real images and landmarks from an online hands dataset (http://www.imm.dtu.dk/pubdb/p.php?403), we infer a transformation between different hands in different poses. We then apply it to the whole source image, creating new hands, whose appearance belongs to the source hands, with geometry akin to the destination hands. We also animate the source image by evaluating $\boldsymbol{\phi}^{\boldsymbol{\theta}}$ for different t's (including outside of $[0, 1]$); *e.g.*, Fig. 7e shows animated frames for the example in the second row of Fig. 7a-7d.

In the no-known-correspondences case, there are two subsets of Ω, denoted U_{src} and U_{dst}, and two feature maps, $I_{\mathrm{src}} : U_{\mathrm{src}} \to \mathcal{F}$ and $I_{\mathrm{dst}} : U_{\mathrm{dst}} \to \mathcal{F}$, where $\mathcal{F}$ is some feature space (*e.g.*, color), and one seeks $T : \Omega \to \Omega$ such that $I_{\mathrm{dst}} \circ T^{-1} \approx I_{\mathrm{src}}$ (or such that $I_{\mathrm{src}} \circ T \approx I_{\mathrm{dst}}$). This is, *e.g.*, the case for time warping (when $n = 1$) and landmark-free image registration ($n > 1$). Figure 8 (left and middle) shows inferred time warps for motion-capture data from [69]. A similar procedure can be applied to 2D images, in which case it is known as image registration. Figure 10 shows some results for registrations of images of digits using a pixelwise Gaussian likelihood model and MCMC inference over CPAB transformations. We used 10K iterations for each single registration which took 10 [sec] (*i.e.*, ~ 0.001 [sec] per each MCMC iteration).

As an example for statistics on the inferred transformations, Fig. 8 (right) shows a 1D warping of $\bar{u}$ (defined in the figure's caption) along the 1^{st} eigen warp (computed using PCA on inferred velocity fields); *i.e.*, we use Euclidean statistics in the linear $\mathcal{V}$ and map the results to the nonlinear M. For a 2D example in the same spirit, we first performed thousands of within-class pairwise registrations of images of digits as described above. Overall we estimated $\sim 30,000$ transformations *i.e.*, $\sim 3,000$ per class (class of 1's, class of 2's, etc.). Next, we computed PCA on the inferred CPA velocity fields; Fig. 11 shows the first eigen wrap of each class. The **Sup. Mat.** includes a **video** showing the result of applying the first 3 eigen warps (per class) applied to 100 random images from the training set.

5.5 Handling a Large Number of Landmarks

In the hand experiment we showed inference for landmark-based warping using a relatively small set of landmarks on real data. The inference technique used there was MAP estimation using the Metropolis algorithm. Here we show, using synthetic data, that the framework can easily uti-

lize a large set of landmark for inference over the latent transformation between the landmark sets; see Fig. 12. The technique used here is ML estimation using the Metropolis algorithm. It is not surprising that with more data the results improve, suggesting empirical evidence that the estimation procedure is *consistent*. However, our point here is that most other methods cannot easily utilize a large set of landmarks.

5.6 Inference over a CPA Field Piecewise w.r.t. t

As a proof-of-concept for the utility of Algorithm 2, we used it to "prettify" the output of a dense-correspondence tool. We extracted dense correspondences, $\{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^{N_{\text{pts}}}$, between two images using SIFT-Flow [20]. Then, we discarded the images and treated the correspondences as *noisy observations* fed as input to Algorithm 2. Finally, for visualization purposes, we used the output of Algorithm 2 to warp the source image. Figure 14 shows a typical result.

5.7 Representing a CDF using a CPAB Transformation

Figure 13 shows results (using Metropolis' algorithm) for fitting $F^{\boldsymbol{\theta}} \triangleq T^{\boldsymbol{\theta}} \circ F_0$, where F_0 is the CDF of the uniform distribution on $J = [-7, 7]$, to a 3-component Gaussian mixture. Note $F^{\boldsymbol{\theta}}$ well approximates the target CDF.

6 Conclusion

We proposed new classes of diffeomorphisms that are simple, expressive, efficient, and have many applications. We showed these objects are evaluated fast and with high accuracy (exact in 1D) and that they are highly expressive. While we required Ω to be a hyperrectangle it is not hard to see that, more generally, Ω may be any finite intersection of closed half spaces of $\mathbb{R}^n$. Like the space in [27], our spaces lack closure under composition; but since $T^{\boldsymbol{\theta}_2} \circ T^{\boldsymbol{\theta}_1} \in G$, our representation can still be used within frameworks that apply consecutive transformations [16]. In fact, Algorithm 2 deals exactly with such scenarios. While this work focused on representation it was primarily motivated by modeling-and-inference considerations: *e.g.*, the ability to quickly run MCMC inference over diffeomorphisms is a hallmark of the method, showing that simplicity pays off.

Acknowledgments

O.F. and J.W.F. are partially supported by U.S. Office of Naval Research MURI program, award N000141110688, and VITALITE, which receives support from U.S. ARO MURI, award W911NF-11-1-0391. S.H. was funded by the Danish Council for Independent Research, Natural Sciences.

References

[1] S. Allassonnière, E. Kuhn, A. Trouvé *et al.*, "Construction of bayesian deformable models via a stochastic approximation algorithm: a convergence study," *Bernoulli*, 2010.
[2] S. Allassonniere, S. Durrleman, and E. Kuhn, "Bayesian mixed effect atlas estimation with a diffeomorphic deformation model," *SIAM Journal on Imaging Sciences*, 2015.
[3] M. Zhang and P. T. Fletcher, "Bayesian statistical shape analysis on the manifold of diffeomorphisms," 2016.
[4] O. Freifeld, S. Hauberg, B. Kayhan, and J. W. Fisher III, "Highly-expressive spaces of well-behaved transformations: Keeping it simple," in *ICCV*, 2015.
[5] S. Hauberg, O. Freifeld, A. Larsen, J. Fisher, and L. Hansen, "Dreaming more data: Class-dependent distributions over diffeomorphisms for learned data augmentation," in *AISTATS*, 2016.
[6] U. Grenander, *General pattern theory: A mathematical study of regular structures*. Clarendon Press, 1993.
[7] U. Grenander and M. Miller, "Representations of knowledge in complex systems," *Journal of the Royal Statistical Society. Series B (Methodological)*, 1994.
[8] P. Dupuis, U. Grenander, and M. I. Miller, "Variational problems on flows of diffeomorphisms for image matching," *QAM*, 1998.
[9] A. Trouvé, "Diffeomorphisms groups and pattern matching in image analysis," *IJCV*, 1998.
[10] S. C. Joshi and M. I. Miller, "Landmark matching via large deformation diffeomorphisms," *IEEE TIP*, 2000.
[11] M. Vaillant, M. I. Miller, L. Younes, and A. Trouvé, "Statistics on diffeomorphisms via tangent space representations," *NeuroImage*, 2004.
[12] M. F. Beg, M. I. Miller, A. Trouvé, and L. Younes, "Computing large deformation metric mappings via geodesic flows of diffeomorphisms," *IJCV*, 2005.
[13] S. Allassonnière, A. Trouvé, and L. Younes, "Geodesic shooting and diffeomorphic matching via textured meshes," in *EMMCVPR*. Springer, 2005.
[14] H. Guo, A. Rangarajan, and S. Joshi, "Diffeomorphic point matching," in *Handbook of Mathematical Models in Computer Vision*. Springer, 2006.
[15] U. Grenander and M. Miller, *Pattern theory: from representation to inference*. Oxford University Press, 2007.
[16] T. Vercauteren, X. Pennec, A. Perchant, and N. Ayache, "Diffeomorphic demons: Efficient non-parametric image registration," *NeuroImage*, 2009.
[17] D. Mumford and A. Desolneux, *Pattern theory: the stochastic analysis of real-world signals*. AK Peters, 2010.
[18] L. Younes, "Spaces and manifolds of shapes in computer vision: An overview," *IVC*, 2012.
[19] S. Durrleman, S. Allassonnière, and S. Joshi, "Sparse adaptive parameterization of variability in image ensembles," 2013.
[20] C. Liu, J. Yuen, and A. Torralba, "Sift flow: Dense correspondence across scenes and its applications," *Pattern Analysis and Machine Intelligence, IEEE Transactions on*, 2011.
[21] J. Kim, C. Liu, F. Sha, and K. Grauman, "Deformable spatial pyramid matching for fast dense correspondences," in *CVPR*, 2013.
[22] T. F. Cootes, C. J. Twining, V. Petrovic, R. Schestowitz, and C. J. Taylor, "Groupwise construction of appearance models using piecewise affine deformations." in *BMVC*, 2005.
[23] A. Fawzi and P. Frossard, "Measuring the effect of nuisance variables on classifiers," 2016.
[24] R. Szeliski, *Computer vision: algorithms and applications*. Springer Science & Business Media, 2010.
[25] M. Galun, T. Amir, T. Hassner, R. Basri, and Y. Lipman, "Wide baseline stereo matching with convex bounded distortion constraints," in *ICCV*, 2015.
[26] D. Lin, E. Grimson, and J. Fisher, "Modeling and estimating persistent motion with geometric flows," in *CVPR*, 2010.
[27] V. Arsigny, O. Commowick, X. Pennec, and N. Ayache, "A log-euclidean polyaffine framework for locally rigid or affine registration," in *BIR*. Springer, 2006.
[28] V. Arsigny, X. Pennec, and N. Ayache, "Polyrigid and polyaffine transformations: a novel geometrical tool to deal with non-rigid deformations–application to the registration of histological slices," *MIA*, 2005.
[29] V. Arsigny, O. Commowick, X. Pennec, and N. Ayache, "A log-euclidean framework for statistics on diffeomorphisms," in *MICCAI*, 2006.
[30] T. Mansi, X. Pennec, M. Sermesant, H. Delingette, and N. Ayache, "ilogdemons: A demons-based registration algorithm for tracking incompressible elastic biological tissues," *IJCV*, 2011.
[31] M. Zhang and P. T. Fletcher, "Finite-dimensional Lie algebras for fast diffeomorphic image registration," in *IPMI*, 2015.
[32] A. Censi and R. M. Murray, "Learning diffeomorphism models of robotic sensorimotor cascades," in *ICRA*, 2012.
[33] A. Censi, A. Nilsson, and R. M. Murray, "Motion planning in observations space with learned diffeomorphism models." in *ICRA*, 2013.
[34] A. Nilsson and A. Censi, "Accurate recursive learning of uncertain diffeomorphism dynamics," 2013.

[35] M. Kilian, N. Mitra, and H. Pottmann, "Geometric modeling in shape space," 2007.
[36] E. S. Gawlik, P. Mullen, D. Pavlov, J. E. Marsden, and M. Desbrun, "Geometric, variational discretization of continuum theories," *Physica D: Nonlinear Phenomena*, 2011.
[37] P. Fletcher, C. Lu, and S. Joshi, "Statistics of shape via principal geodesic analysis on Lie groups," in *CVPR*, 2003.
[38] E. G. Miller and C. Chefd'hotel, "Practical non-parametric density estimation on a transformation group for vision," in *CVPR*, 2003.
[39] F. Porikli, O. Tuzel, and P. Meer, "Covariance tracking using model update based on Lie algebra," in *CVPR*, 2006.
[40] D. Lin, E. Grimson, and J. Fisher, "Learning visual flows: A Lie algebraic approach," in *CVPR*, 2009.
[41] O. Freifeld and M. Black, "Lie bodies: A manifold representation of 3D human shape," in *ECCV*. Springer, 2012.
[42] X. Pennec, "Probabilities and statistics on Riemannian manifolds: Basic tools for geometric measurements," in *NSIP*, 1999.
[43] P. Fletcher, C. Lu, S. Pizer, and S. Joshi, "Principal geodesic analysis for the study of nonlinear statistics of shape," *IEEE TMI*, 2004.
[44] S. Sommer, F. Lauze, S. Hauberg, and M. Nielsen, "Manifold valued statistics, exact principal geodesic analysis and the effect of linear approximation," *ECCV*, 2010.
[45] R. Subbarao and P. Meer, "Nonlinear mean shift over riemannian manifolds," *IJCV*, 2009.
[46] A. Srivastava, S. H. Joshi, W. Mio, and X. Liu, "Statistical shape analysis: Clustering, learning, and testing," *IEEE PAMI*, 2005.
[47] J. Hinkle, P. T. Fletcher, and S. Joshi, "Intrinsic polynomials for regression on riemannian manifolds," *JMIV*, 2014.
[48] M. T. Harandi, M. Salzmann, and R. Hartley, "From manifold to manifold: geometry-aware dimensionality reduction for SPD matrices," in *ECCV*. Springer, 2014.
[49] M. Lorenzi, N. Ayache, and X. Pennec, "Schild's ladder for the parallel transport of deformations in time series of images," in *IPMI*, 2011.
[50] S. Taheri, P. Turaga, and R. Chellappa, "Towards view-invariant expression analysis using analytic shape manifolds," in *AFGR*, 2011.
[51] M. Lorenzi and X. Pennec, "Geodesics, parallel transport & one-parameter subgroups for diffeomorphic image registration," *IJCV*, 2012.
[52] Q. Xie, S. Kurtek, H. Le, and A. Srivastava, "Parallel transport of deformations in shape space of elastic surfaces," in *ICCV*, 2013.
[53] P. Fletcher, "Geodesic regression and the theory of least squares on Riemannian manifolds," *IJCV*, 2012.
[54] S. Jayasumana, R. Hartley, M. Salzmann, H. Li, and M. Harandi, "Kernel methods on the riemannian manifold of symmetric positive definite matrices," in *CVPR*, 2013.
[55] O. Freifeld, S. Hauberg, and M. J. Black, "Model transport: Towards scalable transfer learning on manifolds," in *CVPR*, 2014.
[56] Y. Rubner, C. Tomasi, and L. Guibas, "Metric for distributions with applications to image databases," in *CVPR*, 1998.
[57] A. Srivastava, I. Jermyn, and S. Joshi, "Riemannian analysis of probability density functions with applications in vision," in *CVPR*, 2007.
[58] F. L. Bookstein, "Principal warps: Thin-plate splines and the decomposition of deformations," *IEEE TPAMI*, 1989.
[59] N. Arad, N. Dyn, D. Reisfeld, and Y. Yeshurun, "Image warping by radial basis functions: Application to facial expressions," *CVGIP*, 1994.
[60] C. Liu, H.-Y. Shum, and C.-S. Zhang, "A two-step approach to hallucinating faces: global parametric model and local nonparametric model," in *CVPR*, 2001.
[61] T.-L. Chen and S. Geman, "Image warping using radial basis functions," *JAS*, 2014.
[62] M. Nielsen, P. Johansen, A. Jackson, B. Lautrup, and S. Hauberg, "Brownian warps for non-rigid registration," *JMIV*, 2008.
[63] O. Weber, R. Poranne, and C. Gotsman, "Biharmonic coordinates," in *CGF*, 2012.
[64] T. Nir, A. M. Bruckstein, and R. Kimmel, "Over-parameterized variational optical flow," *IJCV*, 2008.
[65] J. Wulff and M. Black, "Efficient sparse-to-dense optical flow estimation using a learned basis and layers," in *CVPR*, 2015.
[66] C. Moler and C. Van Loan, "Nineteen dubious ways to compute the exponential of a matrix, twenty-five years later," *SIAM review*, 2003.
[67] A. H. Al-Mohy and N. J. Higham, "Computing the action of the matrix exponential, with an application to exponential integrators," *SIAM journal on scientific computing*, 2011.
[68] D. Sun, S. Roth, and M. J. Black, "Secrets of optical flow estimation and their principles," in *CVPR*, 2010.
[69] C. Ionescu, D. Papava, V. Olaru, and C. Sminchisescu, "Human3.6m: Large scale datasets and predictive methods for 3d human sensing in natural environments," *IEEE TPAMI*, 2014.

Oren Freifeld earned his PhD (2013) and ScM (2009) in Applied Mathematics from Brown University, and his MSc (2007) and BSc (2005) in Biomedical Engineering from Tel-Aviv University. He is currently a postdoctoral associate at MIT CSAIL. His main fields of research are computer vision, statistical inference, and machine learning.

Søren Hauberg earned his PhD in Computer Science from the Univ. of Copenhagen in 2011. He has been a visiting scholar at UC Berkeley (2010), and a post doc at the Perceiving Systems department at the Max Planck Institute for Intelligent Systems (2012-2014). He is currently a post doc at the Section for Cognitive Systems, at the technical Univ. of Denmark. He works in the span of geometry and statistics.

Kayhan Batmanghelich earned his PhD (2012) in Electrical and System Engineering from University of Pennsylvania. He is currently a postdoctoral associate at MIT CSAIL. He works in the span of machine learning, medical imaging and computational biology.

John W. Fisher III received his Ph.D. in Electrical and Computer Engineering from the University of Florida (UF), Gainesville, in 1997. He is currently a Senior Research Scientist in MIT CSAIL and affiliated with the Laboratory for Information and Decision Systems, MIT. Prior to joining MIT, he has been affiliated with UF as both a faculty member and graduate student since 1987, during which time he conducted research in the areas of ultrawideband radar for ground penetration and foliage penetration applications, radar signal processing, and automatic target recognition algorithms. His current area of research focus includes information theoretic approaches to signal processing, multimodal data fusion, machine learning, and computer vision.